

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по курсовой работе
по дисциплине
Теоретические основы баз данных
«Каталогизация экзопланет»

Выполнил:

студент группы 5130201/40003

Джабраилов А.В.

Подпись: _____

Принял:

К. тех. наук. доц., доц.

Попов С.Г.

Подпись: _____

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Реферат

Предметная область работы охватывает экзопланеты с их физическими параметрами. Проблема заключается в фрагментации астрономических данных по разрозненным каталогам без единой структуры хранения и межкаталожных связей.

Содержание

Реферат	2
Введение	4
1 Постановка задачи	6
2 Аналитика	7
2.1 Описание предметной области	7
2.2 Процессы	10
2.3 Роли участников	11
2.4 Цели функционирования базы данных	11
2.5 Сущности	12
2.6 ER-диаграмма	16
2.7 Чтение ER-диаграммы	18
2.8 Иерархия сущностей	18
3 Проектирование	20
3.1 Таблицы с описанием таблиц базы данных	20
3.2 Лингвистическое описание схемы базы данных	26
3.3 Диаграммы базы данных	27
3.4 Описание программы генерации данных	31
Приложение А. Исходный тест программы генерации базы данных	44
Приложение Б. Исходный текст программы генерации тестовых данных	49

Введение

Космическая индустрия с каждым годом растёт всё быстрее благодаря улучшению аппаратуры, например, новым моделям телескопов и зондов, в связи с чем количество информации о космосе быстро увеличивается. Чтобы управлять этой информацией, необходим системный подход, основанный на принципах структурированного представления данных.

База данных — это совокупность данных, структурно описывающих один или нескольких связанных между собой объектов. **Система управления базой данных (СУБД)** — это программное обеспечение, предназначенное для поддержки и использования больших массивов данных. Потребность в таких системах и их использовании стремительно растёт [15].

Есть разные модели баз данных:

- иерархическая — присутствует чёткая иерархия, у одного дочернего элемента есть только один родитель;
- сетевая — иерархия, в которой дочерние данные могут иметь нескольких родителей;
- реляционная — в ней данные представлены в виде таблицы, а не дерева, как в предыдущих двух [16].

Основным инструментом взаимодействия с БД являются **SQL-запросы** (*Structured Query Language*) — язык для управления реляционными базами данных. Команды SQL могут быть разбиты на три типа:

- язык определения данных (Data Definition Language, DDL) — создание таблицы;
- язык манипулирования данными (Data Manipulation Language, DML) — ввод и извлечение данных;
- язык управления данными (Data Control Language, DCL) — управление доступом пользователей.

Например, с помощью SQL-запросов можно понять, вокруг звёзд какого типа чаще всего обнаруживают экзопланеты. Это делает SQL необходимым инструментом для аналитики данных [16].

В рамках данного проекта используется реляционная модель базы данных, так как она представляет структуру хранения, не основанную на иерархии, а также предоставляет возможность выполнять гибкие запросы. Более того, выбор работы именно с этой моделью баз данных обоснован наличием большого числа инструментов в языках программирования, позволяющих оперировать ими, что гарантирует возможность модификации БД и СУБД на разных платформах.

ЗАМЕТКИ МНЕ: реляционная БД работает еще и быстро, поэтому мы с ней работаем.

1. Постановка задачи

Цель работы: реализация функционирующей реляционной базы данных для выбранной предметной области и заданных SQL-запросов.

Задачи работы:

1. Проанализировать предметную область.
2. Сформировать цели функционирования базы данных (мин. 3), процессы (мин. 1), перечень ролей (мин. 3), сущностей (мин. 7) и их атрибутов (мин. 4 для каждой сущности).
3. Создать ER-диаграмму и изобразить её в нотации Чена.
4. Сформировать схему связи объектов (мин. 6 объектов).
5. Создать таблицы с описанием таблиц базы данных.
6. Сформулировать лингвистическое описание схемы базы данных.
7. Графически описать схемы базы данных на русском и английском языках.
8. Написать текст программы генерации базы данных.
9. Сгенерировать базу данных, содержащую не менее 250 тыс. записей.
10. Реализовать персональные SQL-запросы.
11. Сформировать последовательность выполнения запросов.

2. Аналитика

2.1. Описание предметной области

Экзопланеты — это космические тела, идентифицируемые как планеты, находящиеся за пределами солнечной системы [1]. Чтобы тело считалось планетой, оно должно удовлетворять трём критериям [2]:

- оно должно вращаться вокруг звезды;
- оно должно быть достаточно большим, чтобы силой гравитации поддерживать сферическую форму;
- его гравитация не должна позволять другим объектам такого же размера приближаться к его орбите.

Однако некоторые космические тела, не вращающиеся вокруг какой-либо звезды, так же могут быть отнесены к планетам — их называют планетами-изгоями [1].

Большинство обнаруженных экзопланет расположены в относительно малой области нашей галактики, а именно в пределах тысяч световых лет от Солнечной системы. Например, ближайшая известная экзопланета, *Proxima Centauri b*, находится на расстоянии 4 световых лет от нас. Планет в космосе больше, чем звёзд, однако задача их обнаружения гораздо сложнее, чем со звёздами, так как они не являются самосветящимися объектами, поэтому обнаружить экзопланету можно только косвенно, опираясь на звезду рядом с ней (например, заметив, что часть света звезды перекрывается каким-то телом или что звезда "покачивается" от гравитации другого тела, вращающегося вокруг него).

Каталогизация экзопланет представляет собой многоэтапный процесс сбора, верификации и систематизации данных о них. С момента открытия первой подтверждённой экзопланеты в 1992 году их число превысило 6000 объектов [1], и с каждым годом темпы открытий ускоряются благодаря новым космическим миссиям и телескопам, в том числе тем, что расположены на орбите, а так же зондам. Однако рост объёма данных сопровождается фрагментацией информации: одни и те же планеты могут фигурировать в разных каталогах с разными параметрами, а сами параметры нередко противоречат

друг другу из-за различий в методах наблюдений и обработки или разных дат последнего обновления информации.

Основными участниками процесса каталогизации выступают астрономы-наблюдатели, астрофизики и кураторы каталогов. Наблюдатели регистрируют космические объекты с помощью телескопов, таких как космический телескоп «Кеплер», «Тесс» или наземные обсерватории вроде Ла-Силья в Чили [3]. Полученные данные передаются астрофизикам, которые интерпретируют их для расчёта физических параметров: массы, радиуса, орбитального периода и температуры поверхности. Завершающим этапом становится работа куратора каталога, который проверяет соответствие данных принятым стандартам, устраняет дубликаты и присваивает объекту уникальный идентификатор. В современных каталогах часть задач куратора выполняют автоматизированные системы, однако окончательная верификация по-прежнему требует участия человека.

На сегодняшний день существует несколько независимых каталогов экзопланет, каждый со своей структурой хранения и правилами идентификации. Крупнейший из них — *NASA Exoplanet Archive* [4] — содержит данные о 6000 экзопланетах с изображениями. Альтернативный *Open Exoplanet Catalogue* [5] содержит на 1000 экзопланет меньше, чем у NASA. Европейский архив *Catalogue of Exoplanets* фокусируется на публикациях в рецензируемых журналах. Отсутствие единого формата приводит к тому, что одна и та же экзопланета может иметь разные характеристики в разных каталогах. Это обосновано тем, что при исследовании экзопланет сложно дать точную характеристику всем их параметрам: даже у первой открытой экзопланеты Kepler-186f не известны точные масса, состав и т.д. [7].

Качество данных в каталоге определяется несколькими факторами. Во-первых, методом обнаружения: транзитный метод даёт точные значения радиуса, но не массы; метод радиальных скоростей, напротив, позволяет определить массу, но не размер. Во-вторых, количеством независимых подтверждений: планета, обнаруженная двумя разными телескопами, считается более надёжной. В-третьих, датой последнего измерения: параметры некоторых экзопланет уточняются спустя годы после открытия по мере накопления новых наблюдений.

Существуют разные способы, активно используемые учёными для от-

крытия экзопланет [8].

- **Метод транзитной скорости:** когда планета оказывается прямо между наблюдателем и звездой, вокруг которой она вращается, она частично заслоняет свет звезды. На короткое время свет звезды становится тусклее. Это незначительное изменение, но его достаточно, чтобы астрономы поняли, что вокруг далекой звезды вращается экзопланета. Этот метод называется транзитным. Например, так был обнаружен газовый гигант *51 Pegasi b*. [8]
- **Метод радиальной скорости:** из-за гравитационного воздействия планет звезды раскачиваются в пространстве, что влияет на спектр света, который видят астрономы при наблюдении за звездой. На звезды влияет гравитационное притяжение планет, которые вращаются вокруг них, и при наблюдении в телескоп это влияет на спектр света звезды. Если звезда движется в направлении наблюдателя, ее спектр смещается в сторону синего. Если звезда удаляется от наблюдателя, ее спектр смещается в сторону красного. Этот метод наблюдения называется методом измерения лучевой скорости.
- **Транзитная стереоскопия:** этот метод помогает изучать атмосферу экзопланет. После того как свет уловлен, его можно исследовать, чтобы определить состав атмосфер экзопланет. Это работает, как призма: если направить на нее белый свет, он разделится на радужный спектр. Ученые могут считывать цветовые полосы этого спектра, чтобы определить, какие молекулы в нем присутствуют. Так, например, у газового гиганта *WASP-107b* обнаружили гелий в атмосфере — это первый случай, когда этот элемент был найден в атмосфере экзопланет [8].

Экзопланеты классифицируются по физическим характеристикам на несколько типов [10].

- **газовые гиганты** — массивные планеты, обычно размером с Сатурн или Юпитер, но могут быть и больше. В этой категории планет есть подкатегории, например, горячие юпитеры — газовые гиганты, вращающиеся вокруг звезды так быстро, что их температура достигает тысяч градусов Цельсия;

- **нептуновые планеты** размером с Нептун или Уран. Состав у них разнообразный, но все имеют атмосферу, преимущественно состоящую из водорода и гелия, а также имеют твёрдое ядро. Также существует подкатегория мини-Нептунов — планет меньше Нептуна и больше Земли, но в Солнечной системе таких нет;
- **суперземли** — каменные планеты, массой больше Земли и меньше Нептуна; атмосфера у них может как быть, так и отсутствовать;
- **планеты земной группы** — это планеты размером с Землю или меньше, состоящие из горных пород, силикатов (солей кремниевых кислот), воды или углерода. Дальнейшие исследования покажут, есть ли на некоторых из них атмосфера, океаны или другие признаки обитаемости.

Создание унифицированной базы данных позволило бы решить ключевые проблемы современной каталогизации: устранить дублирование записей, обеспечить отслеживание истории изменений параметров и предоставить исследователям инструмент для поиска объектов по комбинации критериев — например, «все каменные планеты в зоне обитаемости с массой от 0,8 до 1,5 массы Земли». Такая система особенно востребована в эпоху новых миссий, таких как космический телескоп «Джеймс Уэбб» [11], запущенный в 2021 году, благодаря которым получится открыть множество других экзопланет и дать более точную оценку уже известным [8].

2.2. Процессы

В данной предметной области имеются три процесса.

1. Процесс каталогизации экзопланеты. Астроном-наблюдатель обнаруживает космическое тело и передаёт информацию астрофизику, определяющему физические параметры этого тела. Данные передаются куратору каталога (архивариусу), который проверяет соответствие измерений стандартам и присваивает экзопланете идентификатор в соответствии с принятым в каталоге правилом.

2. Процесс обновления данных об экзопланете. Обновление данных об экзопланете предполагает циклическое наблюдение её положения и состояния наблюдателем, вычисление текущих параметров и обнаружение различий с известными данными, после чего куратор каталога вносит скорректированные значения в каталог с указанием даты последнего измерения. Сегодня функцию архивариуса нередко выполняют автоматизированные системы.

3. Процесс извлечения данных об экзопланете. Астроном, астрофизик, куратор каталога или астроном-любитель может извлечь данные об экзопланете из каталога, например, чтобы изучить её или сравнить данные с другими.

2.3. Роли участников

В вышеперечисленных процессах присутствуют следующие участники:

- астроном-наблюдатель — специалист, осуществляющий регистрацию космических объектов с помощью телескопов и регистрирующий первичные данные об их положении и движении;
- астрофизик — учёный, интерпретирующий наблюдательные данные для определения физических параметров экзопланет: массы, радиуса, температуры и орбитальных характеристик;
- куратор каталога — ответственный за верификацию поступающих данных, присвоение уникальных идентификаторов и соблюдение форматов хранения в соответствии со стандартами каталога;
- астроном-любитель — непрофессиональный исследователь, использующий открытые данные для самостоятельного изучения экзопланет и потенциального участия в citizen science-проектах [14];

2.4. Цели функционирования базы данных

Цели функционирования базы данных следующие:

- хранение данных о наблюдателях;

- хранение данных о первооткрывателях;
- хранение информации о наблюдении за экзопланетами;
- хранение использования телескопов наблюдателями;
- хранение принадлежности экзопланеты к более крупным космическим структурами.

2.5. Сущности

В данной предметной области присутствуют сущности, перечисленные ниже.

- Экзопланета — планета, находящаяся за пределами Солнечной системы. Термин и критерии, по которым определяется, является ли космическое тело планетой, даны в разделе Описание предметной области. Они обладают следующими атрибутами:
 - уникальный идентификатор — строка длиной до 100 символов, непустая, уникальная, состоящая из латинских букв, цифр, плюсов, минусов, дефисов, пробелов и точек, есть чувствительность к регистру (например, *Kepler-186f*) [21];
 - масса в массах Земли ($M_{\oplus}$) — вещественное число, > 0 , с точностью до 4 знаков после запятой;
 - радиус в радиусах Земли ($R_{\oplus}$) — вещественное число, > 0 , с точностью до 4 знаков после запятой;
 - орбитальный период в земных сутках (сут.) вещественное число, > 0 , с точностью до 4 знаков после запятой;
 - тип планеты — перечисление: {Gas Giant, Neptunian, Superearth, Terrestrial};
 - расстояние от нас в парсеках — вещественное число, ≥ 0 , с точностью до 7 знаков после запятой;
 - дата открытия — дата в формате ГГГГ-ММ-ДД, 1992-01-01 – текущая дата [20].

- дата последнего наблюдения — календарная дата в формате ГГГГ-ММ-ДД, дата открытия — текущая дата.
- Звезда — самосветящийся газовый шар, удерживаемый гравитацией, в котором происходят термоядерные реакции синтеза водорода в гелий. Обладает следующими атрибутами:
 - уникальный идентификатор — строка длиной до 100 символов, непустая, уникальная, состоящая из латинских букв, цифр, плюсов, минусов дефисов, пробелов и точек, есть чувствительность к регистру (например, *Proxima Centauri*);
 - спектральный класс — перечисление: $\{O, B, A, F, G, K, M, W, L, D\}$ [19]
 - масса в массах Солнца ($M_{\odot}$) — вещественное число, > 0 , с точностью до 3 знаков после запятой;
 - радиус в радиусах Солнца ($R_{\odot}$) — вещественное число, > 0 , с точностью до 3 знаков после запятой;
 - температура поверхности в Кельвинах (K) — целое число > 0 ;
 - расстояние от нас в парсеках — вещественное число, ≥ 0 , с точностью до 7 знаков после запятой.
- Планетарная система — совокупность планет и других небесных тел, гравитационно связанных и обращающихся вокруг одной звезды или звёздной системы. Обладает следующими атрибутами:
 - уникальный идентификатор — строка длиной до 100 символов, непустая, уникальная, состоящая из латинских букв, цифр, плюсов, минусов, дефисов, пробелов и точек, есть чувствительность к регистру (например, *TRAPPIST-1*);
 - количество подтверждённых планет — целое число, ≥ 1 ;
 - количество планет в зоне обитаемости — целое число, ≥ 0 ;
 - расстояние от нас в парсеках — вещественное число, ≥ 0 , с точностью до 7 знаков после запятой.

- Звёздная система — группа из одной или нескольких звёзд, связанных взаимным гравитационным притяжением и движущихся вокруг общего центра масс.

Обладает следующими атрибутами:

- уникальный идентификатор — строка длиной до 100 символов, непустая, уникальная, состоящая из латинских букв, цифр, плюсов, минусов, дефисов, пробелов и точек, есть чувствительность к регистру (например, *TOI-1338*);
- количество звёзд в звёздной системе — целое число, ≥ 1 ;
- количество подтверждённых планет — целое число, ≥ 0 ;
- количество планет в зоне обитаемости — целое число, ≥ 0 ;
- расстояние от нас в парсеках — вещественное число, ≥ 0 , с точностью до 7 знаков после запятой.

- Звёздное скопление — группа звёзд, имеющих общее происхождение и удерживаемых вместе гравитацией.

Обладает следующими атрибутами:

- уникальный идентификатор — строка длиной до 100 символов, непустая, уникальная, состоящая из латинских букв, цифр, плюсов, минусов, дефисов, пробелов и точек, есть чувствительность к регистру (например, *M4 [17]*);
- количество звёзд в звёздном скоплении — целое число, ≥ 10 ;
- количество подтверждённых планет — целое число, ≥ 0 ;
- расстояние от нас в парсеках — вещественное число, ≥ 0 , с точностью до 7 знаков после запятой.

- Галактика — гигантская гравитационно связанная система из звёзд, газа, пыли и тёмной материи, имеющая характерную форму и структуру.

Обладает следующими атрибутами:

- уникальный идентификатор — строка длиной до 100 символов, непустая, уникальная, состоящая из латинских букв, цифр, плюсов, минусов, дефисов, пробелов и точек, есть чувствительность к регистру (например, *The Milky Way*);

- количество подтверждённых планет — целое число, ≥ 0 ;
 - количество планет в зоне обитаемости — целое число, ≥ 0 ;
 - расстояние от нас в парсеках — вещественное число, ≥ 0 , с точностью до 7 знаков после запятой (для Млечного Пути = 0).
- Телескоп — инструмент, с помощью которого исследуют космос. В данном случае это телескоп, с помощью которого была обнаружена экзопланета, например, для *WASP-107b* это *Hubble*.

Обладает следующими атрибутами:

- название телескопа — строка длиной до 50 символов, непустая, уникальная, состоящая из латинских букв, цифр, дефисов, пробелов и точек, чувствительность к регистру (например, *Hubble*);
 - тип — перечисление: {космический, наземный};
 - диаметр апертуры в метрах — вещественное число, > 0 , с точностью до 2 знаков после запятой;
 - год ввода в эксплуатацию — нижняя граница — 1609 — текущая дата;
 - организация-оператор — непустая строка длиной до 50 символов (например, *NASA*);
 - количество планет, открытых телескопом — целое число, ≥ 0 .
- Наблюдатель — астроном, наблюдающий за экзопланетами, или организация-оператор телескопа, которая наблюдает за экзопланетами, так как не все планеты наблюдаются вручную и только одним человеком.

Обладает следующими атрибутами:

- уникальный идентификатор — ФИО человека или название организации-оператора телескопа; строка длиной до 100 символов, непустая, уникальная, состоящая из латинских букв, цифр, дефисов, пробелов и точек, чувствительность к регистру (например, {NASA});
- дата рождения / основания — нижняя граница — 1609 — текущая дата;

- страна — словарь, включающий в себя все страны;
- количество открытий — целое число, ≥ 0 .

2.6. ER-диаграмма

Визуальное представление структуры базы данных, отображающее сущности, их атрибуты и связи между ними в виде ER-диаграммы представлено на рис. 1.

Цели функционирования базы данных следующие:

- добавление и актуализация данных об экзопланетах и связанных с ними астрономических объектов и групп,
- обеспечение долговременного хранения этих данных,
- предоставление возможности поиска объектов по параметрам,
- фиксация истории изменений измеренных параметров при повторных наблюдениях за экзопланетой,
- хранение данных о первооткрывателях.

Чтение ER-диаграммы

- Наблюдатель Оперирует Телескопами.
- Экзопланеты Обнаруживаются Телескопами.
- Экзопланеты Обращаются [вокруг] Звёзд.
- Звезды Притягивают Планетарную систему.
- Планетарная система Консолидирует Экзопланеты.
- Планетарная система Кластеризуется [в] Звёздную систему.
- Звёздные системы Группируются в Звёздное скопление.
- Звёздные скопления Концентрируются в Галактику

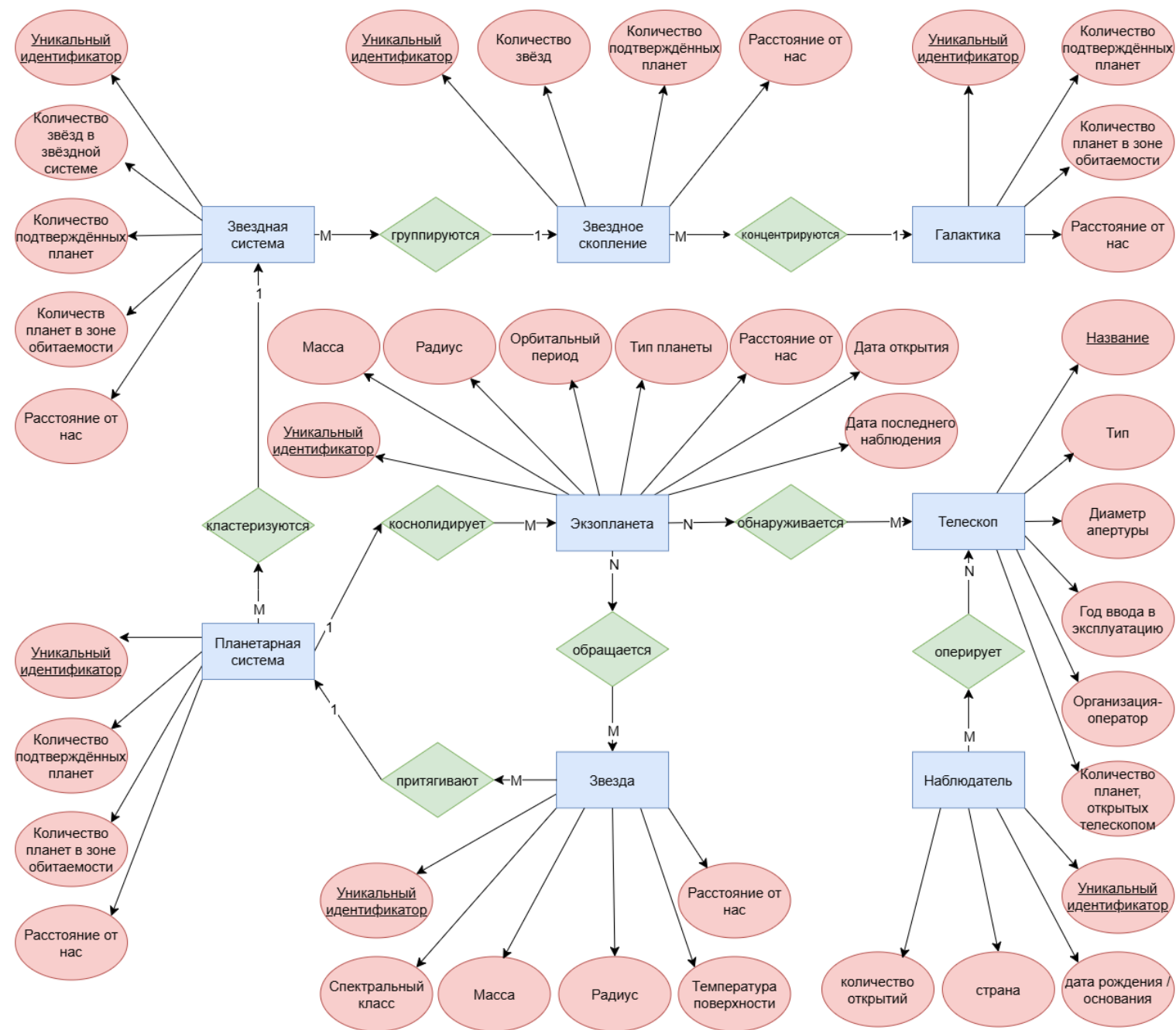


Рис. 1: ER-диаграмма

2.7. Чтение ER-диаграммы

- Наблюдатель Оперировать Телескопами.
- Экзопланеты Обнаруживаются Телескопами.
- Экзопланеты Обращаются [вокруг] Звёзд.
- Звезды Притягивают Планетарную систему.
- Планетарная система Консолидирует Экзопланеты.
- Планетарная система Кластеризуется [в] Звездную систему.
- Звёздные системы Группируются в Звёздное скопление.
- Звёздные скопления Концентрируются в Галактику.

2.8. Иерархия сущностей

Сущности рассматриваемой предметной области можно представить в виде следующей иерархической модели (рис. 2):

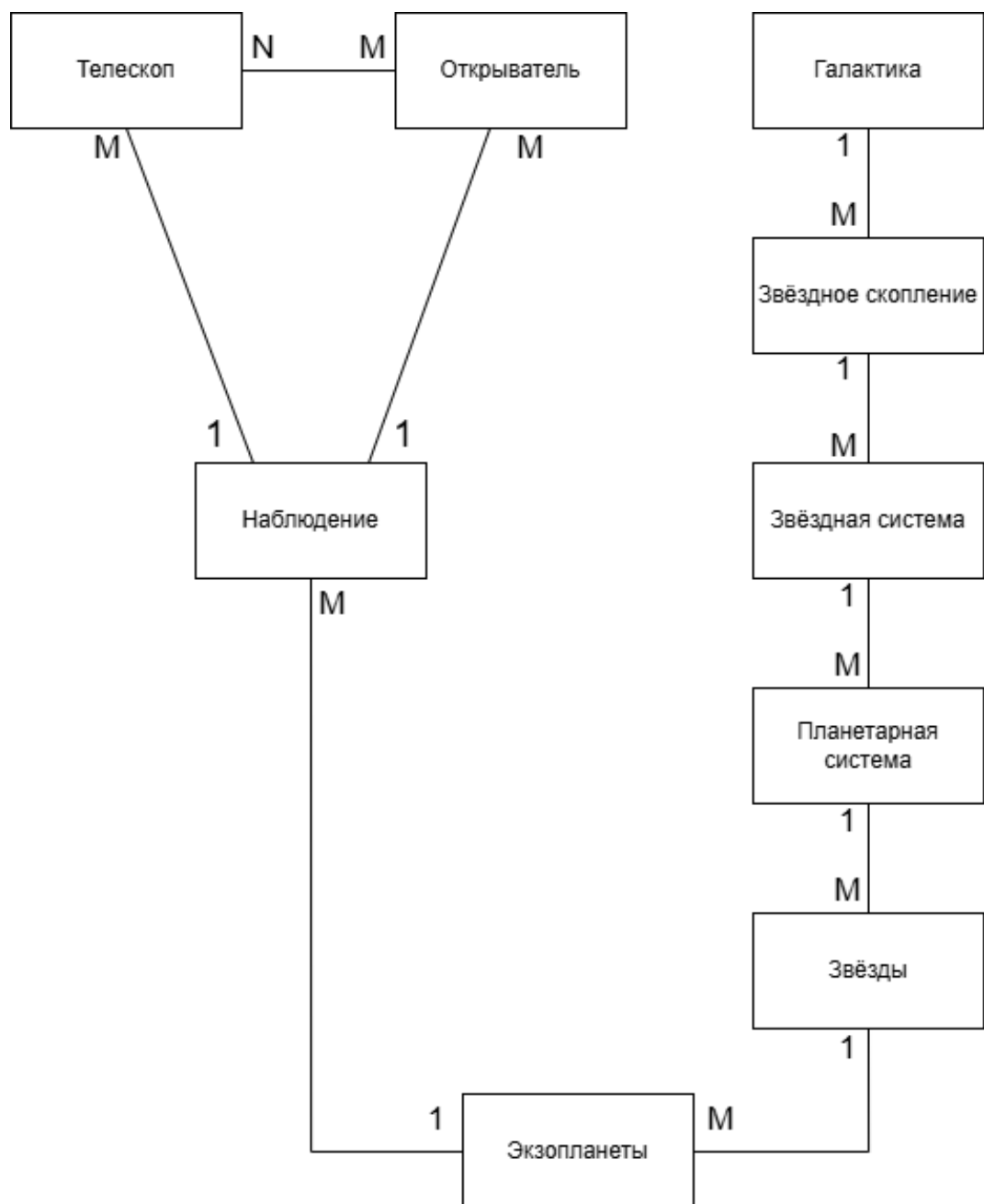


Рис. 2: ER-диаграмма

3. Проектирование

3.1. Таблицы с описанием таблиц базы данных

В п. 1.4 были перечислены сущности, их атрибуты и домены.

В следующих таблицах указаны метаданные полученной базы данных и обоснования доменов атрибутов.

Таблица 1: Экзопланета

№	Название RU	Название EN	Тип атрибута	Тип ключа	Ссылка	Прим.
1	экзопланета_id	exo_id	VARCHAR(30)	PK	-	NOT NULL
2	масса	mass	DECIMAL(10,4)	-	-	NOT NULL
3	радиус	radius	DECIMAL(10,4)	-	-	NOT NULL
4	орбитальный_период	orb_period	DECIMAL(10,4)	-	-	NOT NULL
5	тип_экзопланеты	exo_type	ENUM	-	-	NOT NULL
6	расстояние	dist	DECIMAL(12,6)	-	-	NOT NULL
7	дата_открытия	discov_date	DATE	-	-	NOT NULL
8	дата_последнего_наблюдения	last_obs_date	DATE	-	-	NOT NULL
9	система_id	sys_id	VARCHAR(30)	FK	Планетарная_система	NOT NULL (system_id)

- Идентификатор экзопланеты, звезды, планетарной системы, звёздной системы, звёздного скопления и галактики ограничен 30 символами, а названия космических объектов могут содержать буквенно-цифровые обозначения с дефисами и точками (например, *Kepler-186f*, *Proxima Centauri*). Это обосновано тем, что самое длинное название звезды — *SDSS J122906.70+112221.4* (24 символа), а названия экзопланет представляют собой название звезды + идентификатор экзопланеты, кот. может быть не более 5 символов, + 1 пробел, итого 30 символов.
- Масса, радиус и орбитальный период экзопланеты хранятся как *DECIMAL(10,4)*.
- Расстояние от нас до экзопланеты хранится в парсеках как *DECIMAL(12,6)* для обеспечения точности до 6 знаков после запятой, так как расстояния в световых годах указываются с точностью не более 2 знаков после запятой (до ближайшего объекта, расстояние до которого известно с наибольшей точностью, — *Proxima Centauri*

b — расстояние составляет 4,24 светового года), парсек в световых годах указан с точностью до 4 знаков после запятой, а результат их умножения будет содержать максимум 6 знаков после запятой.

- Дата открытия и дата последнего наблюдения хранятся как *DATE* в формате ГГГГ-ММ-ДД. Минимальная дата обоснована датой открытия первой экзопланеты (1992-01-01). Максимальная дата обоснована тем, что объект, не открытый сейчас, не является открытым в принципе, потому эта дата не может превышать сегодняшнюю дату.
- Типы экзопланет хранятся как *ENUM* с фиксированным набором значений: {Gas Giant, Neptunian, Superearth, Terrestrial}.

Таблица 2: Звезда

№	Название RU	Название EN	Тип атрибута	Тип ключа	Ссылка	Прим.
1	звезда_id	star_id	VARCHAR(30)	PK	-	NOT NULL
2	класс	class	ENUM	-	-	NOT NULL
3	масса	mass	DECIMAL(5,2)	-	-	NOT NULL
4	радиус	radius	DECIMAL(10,4)	-	-	NOT NULL
5	температура	temp	INT	-	-	NOT NULL
6	расстояние	dist	DECIMAL(16,6)	-	-	NOT NULL
7	система_id	sys_id	VARCHAR(30)	FK	Планетарная_система_id(system_id)	NOT NULL

- Масса звезды хранится как *DECIMAL(5,2)*, т.к. самая массивная звезда *R136a1* имеет массу 150–200 масс Солнца (т.е. 3 целых числа), а двух знаков после запятой достаточно для точности размеров ближайших звезд.
- Расстояние до звезды хранится как *DECIMAL(16,6)* — 10 целых знаков (самая далекая звезда находится на расстоянии 8.6 млрд парсек), 6 знаков после запятой для точности (так же, как с экзопланетами).
- Температура поверхности звезды хранится как целое число (*INT*), так как в астрономических каталогах температура указывается без дробной части.

- Спектральный класс звезды хранится как *VARCHAR(5)* для представления комбинированного класса (самый длинный класс может состоять из пяти символов, например, *M5III*).

Таблица 3: Планетарная система

№	Название RU	Название EN	Тип атрибута	Тип ключа	Ссылка	Прим.
1	система_id	system_id	VARCHAR(30)	PK	-	NOT NULL
2	подтверждённых планет	confirmed_plan	INT	-	-	NOT NULL
3	планет в зоне обитаемости	habitable_plan	INT	-	-	NOT NULL
4	расстояние	distance	DECIMAL(12,6)	-	-	NOT NULL
5	звёздная_система_id	stell_sys_id	VARCHAR(30)	FK	Звёздная система (stellar_id)	NOT NULL

- Количество подтверждённых планет, планет в зоне обитаемости хранится как целое число (*INT*) и имеют минимальные значения, соответствующие данным астрономическим объектам.

Таблица 4: Звёздная система

№	Название RU	Название EN	Тип атрибута	Тип ключа	Ссылка	Прим.
1	звёздная_система_id	stellar_id	VARCHAR(30)	PK	-	NOT NULL
2	количество звёзд	stars_count	INT	-	-	NOT NULL
3	подтверждённых планет	confirmed_plan	INT	-	-	NOT NULL
4	планет в зоне обитаемости	habitable_plan	INT	-	-	NOT NULL
5	расстояние	dist	DECIMAL(12,6)	-	-	NOT NULL
6	кластер_id	cluster_id	VARCHAR(30)	FK	Кластер (cluster_id)	NOT NULL

- Количество подтверждённых планет, планет в зоне обитаемости, звёзд хранится как целое число (*INT*) и имеют минимальные значения, соответствующие данным астрономическим объектам.

Таблица 5: Кластер

№	Название RU	Название EN	Тип атрибута	Тип ключа	Ссылка	Прим.
1	кластер_id	cluster_id	VARCHAR(30)	PK	-	NOT NULL
2	количество_звёзд	stars_count	INT	-	-	NOT NULL
3	подтверждённых планет	conf_plan	INT	-	-	NOT NULL
4	планет в зоне обитания	hab_plan	INT	-	-	NOT NULL
5	расстояние	dist	DECIMAL(12,6)	-	-	NOT NULL
6	галактика_id	galaxy_id	VARCHAR(30)	FK	Галактика(galaxy_id)	NOT NULL

- Количество подтверждённых планет, звёзд хранится как целое число (*INT*) и имеют минимальные значения, соответствующие данным астрономическим объектам (например, звёздное скопление содержит как минимум 10 звёзд по определению, поэтому это число является минимальным значением данного поля для звёздных скоплений).

Таблица 6: Галактика

№	Название RU	Название EN	Тип атрибута	Тип ключа	Ссылка	Прим.
1	галактика_id	galaxy_id	VARCHAR(30)	PK	-	NOT NULL
2	подтверждённых планет	conf_plan	INT	-	-	NOT NULL
3	планет в зоне обитания	hab_plan	INT	-	-	NOT NULL
4	расстояние	dist	DECIMAL(12,6)	-	-	NOT NULL

- Количество подтверждённых планет и планет в зоне обитаемости хранится как целое число (*INT*) и имеют минимальные значения, соответствующие данным астрономическим объектам.

Таблица 7: Телескоп

№	Название RU	Название EN	Тип атрибута	Тип ключа	Ссылка	Прим.
1	телескоп_id	tele_id	VARCHAR(47)	PK	-	NOT NULL
2	тип телескопа	tele_type	ENUM	-	-	NOT NULL
3	апертура	aper	DECIMAL(6,2)	-	-	NOT NULL
4	дата создания	com_year	INT	-	-	NOT NULL
5	оператор	oper	VARCHAR(81)	-	-	NOT NULL
6	открыто планет	discov_plan	INT	-	-	NOT NULL

- Название телескопа ограничено 47 символами. Самое длинное название телескопа: *Atacama Large Millimeter/submillimeter Array*.
- Диаметр апертуры телескопа хранится как *DECIMAL(6,2)* с точностью до 2 знаков после запятой.
- Год ввода в эксплуатацию хранится как целое число (*INT*) в диапазоне от 1609 до текущего года.
- Организация-оператор телескопа ограничена 81 символами. Самое длинное название организации-оператора телескопа: *European Southern Observatory for Astronomical Research in the Southern Hemisphere*.
- Типы телескопов хранятся как *ENUM*, их значения: {космический, наземный}.

Таблица 8: Наблюдатель

№	Название RU	Название EN	Тип атрибута	Тип ключа	Ссылка	Прим.
1	наблюдатель_id	observer_id	VARCHAR(100)	PK	-	NOT NULL
2	дата_рождения	bday	DATE	-	-	NOT NULL
3	страна	country	ENUM	-	-	NOT NULL
4	открытий	discovs	INT	-	-	NOT NULL

- Название наблюдателя (ФИО или организация) ограничено 100 символами.
- Страна наблюдателя ограничена 56 символами. Самое длинное название страны: *The United Kingdom of Great Britain and Northern Ireland*.

Таблица 9: Наблюдение

№	Название RU	Название EN	Тип атрибута	Тип ключа	Ссылка	Прим.
1	наблюдение_id	observation_id	VARCHAR(30)	PK	-	NOT NULL
2	экзопланета_id	exo_id	VARCHAR(30)	FK	Экзопланета(exo_id)	NOT NULL
3	телескоп_id	tele_id	VARCHAR(47)	FK	Телескоп(tele_id)	NOT NULL
4	наблюдатель_id	observer_id	VARCHAR(100)	FK	Наблюдатель(observer_id)	NOT NULL
5	дата_наблюдения	obs_date	DATE	-	-	NOT NULL

- Идентификатор наблюдения хранится как *VARCHAR(30)* и представляет собой искусственный ключ. В отличие от других сущностей, наблюдение является событием, которое не имеет устойчивого естественного идентификатора, поэтому использование составного ключа (экзопланта, телескоп, наблюдатель, дата) является избыточным и усложняет структуру базы данных. Ограничение в 30 символов выбрано для унификации с другими идентификаторами.

3.2. Лингвистическое описание схемы базы данных

1. Галактика (галактика_id(VARCHAR(30)), подтверждённых_планет(INT), планет_в_зоне_обитаемости(INT), расстояние(DECIMAL(12, 6))).
2. Кластер (кластер_id(VARCHAR(30)), количество_звёзд(INT), подтверждённых_планет(INT), планет_в_зоне_обитаемости(INT), расстояние(DECIMAL(12, 6)), галактика_id(VARCHAR(30))).
3. Звёздная система (звёздная_система_id(VARCHAR(30)), количество_звёзд(INT), подтверждённых_планет(INT), планет_в_зоне_обитаемости(INT), расстояние(DECIMAL(12, 6)), кластер_id(VARCHAR(30))).
4. Планетарная система (система_id(VARCHAR(30)), подтверждённых_планет(INT), планет_в_зоне_обитаемости(INT), расстояние(DECIMAL(12, 6)), звёздная_система_id(VARCHAR(30))).
5. Класс звезды (класс_id(SERIAL), класс(VARCHAR(5))).
6. Звезда (звезда_id(VARCHAR(30)), класс_id(INT), масса(DECIMAL(5, 2)), радиус(DECIMAL(10, 4)), температура(INT), расстояние(DECIMAL(16, 6)), система_id(VARCHAR(30))).
7. Тип экзопланеты (тип_экзопланеты_id(SERIAL), тип_экзопланеты(VARCHAR(30))).
8. Экзопланета (экзопланета_id(VARCHAR(30)), масса(DECIMAL(10, 4)), радиус(DECIMAL(10, 4)), орбитальный_период(DECIMAL(10, 4)), тип_экзопланеты_id(INT), расстояние(DECIMAL(12, 6)), дата_открытия(DATE), дата_последнего_наблюдения(DATE), система_id(VARCHAR(30))).
9. Тип телескопа (тип_телескопа_id(SERIAL), тип_телескопа(VARCHAR(30))).
10. Телескоп (телескоп_id(VARCHAR(47)), тип_телескопа_id(INT), апертура(DECIMAL(6, 2)), дата_создания(INT), оператор(VARCHAR(81)), открыто_планет(INT)).

11. Страна (страна_id(SERIAL), страна(VARCHAR(60))).
12. Наблюдатель (наблюдатель_id(VARCHAR(100)), да-
та_рождения(DATE), страна_id(INT), открытий(INT)).
13. Наблюдение (наблюдение_id(VARCHAR(30)), экзоплане-
та_id(VARCHAR(30)), телескоп_id(VARCHAR(47)), наблюда-
тель_id(VARCHAR(100)), дата_наблюдения(DATE)).

3.3. Диаграммы базы данных

На рис. 3 изображена схема базы данных на русском языке.

На рис. 4 изображена схема базы данных на русском языке.

На рис. 5 изображён порядок заполнения таблиц базы данных.

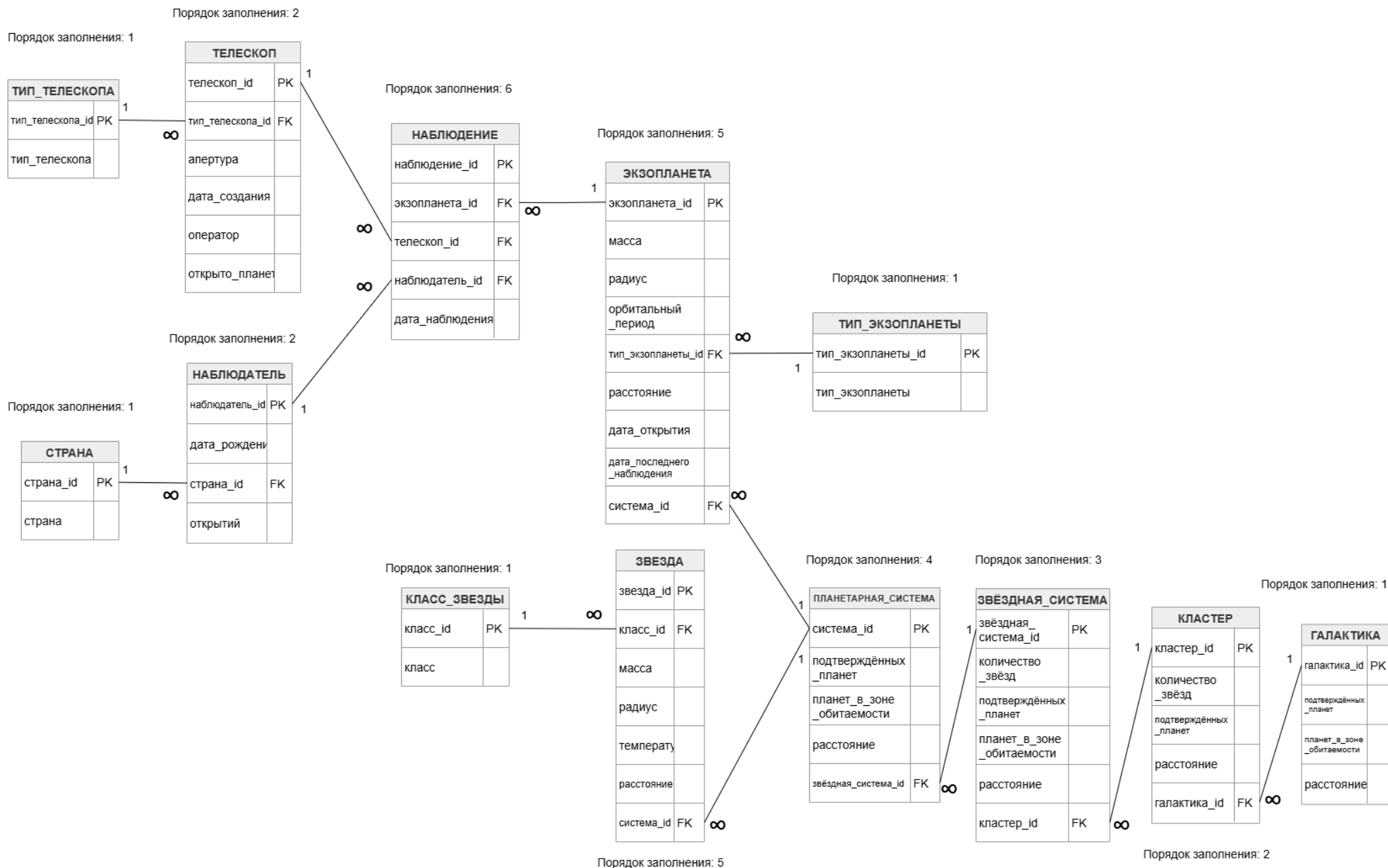


Рис. 3: Диаграмма таблиц на русском языке.

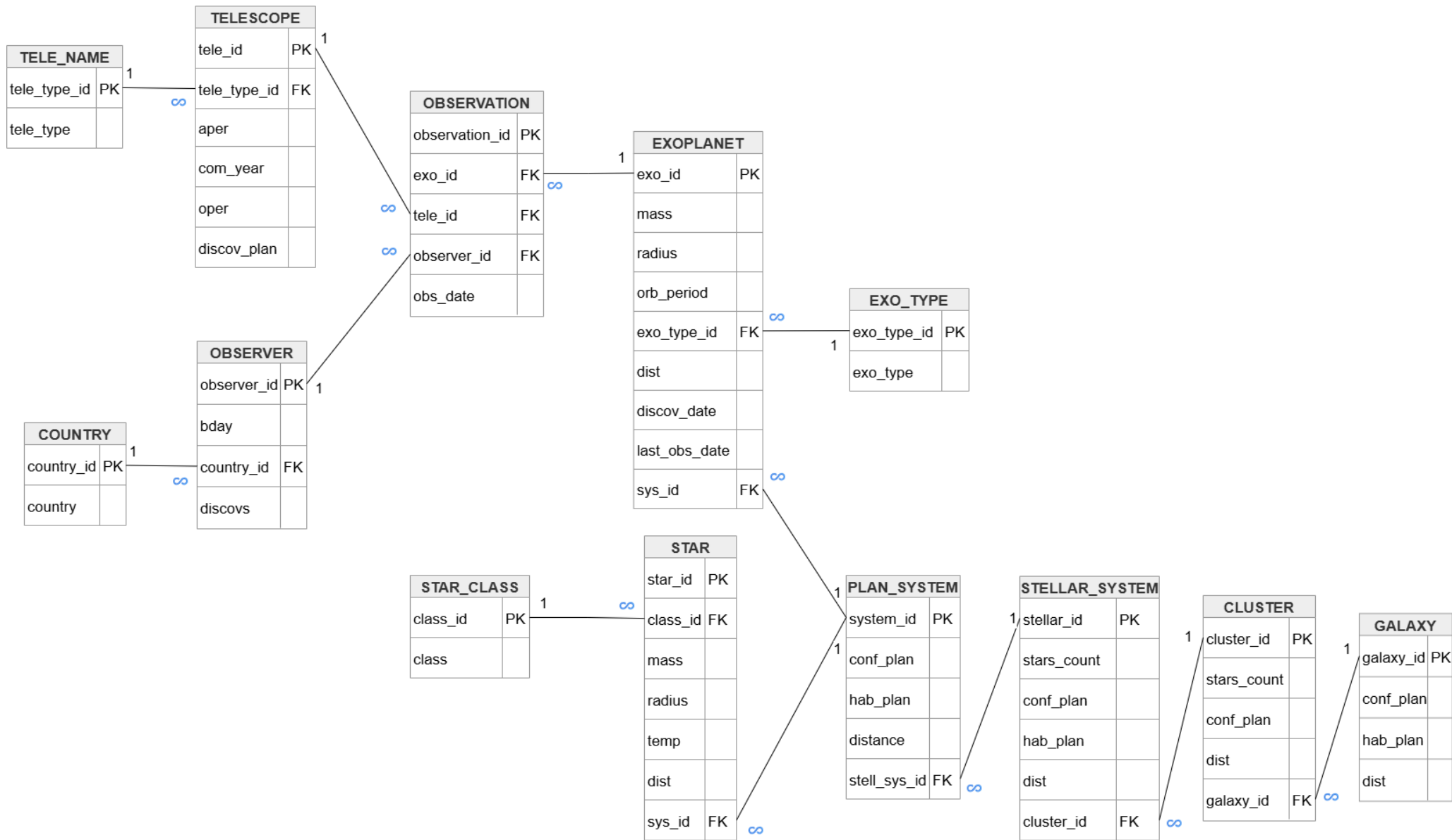


Рис. 4: Диаграмма таблиц на английском языке.

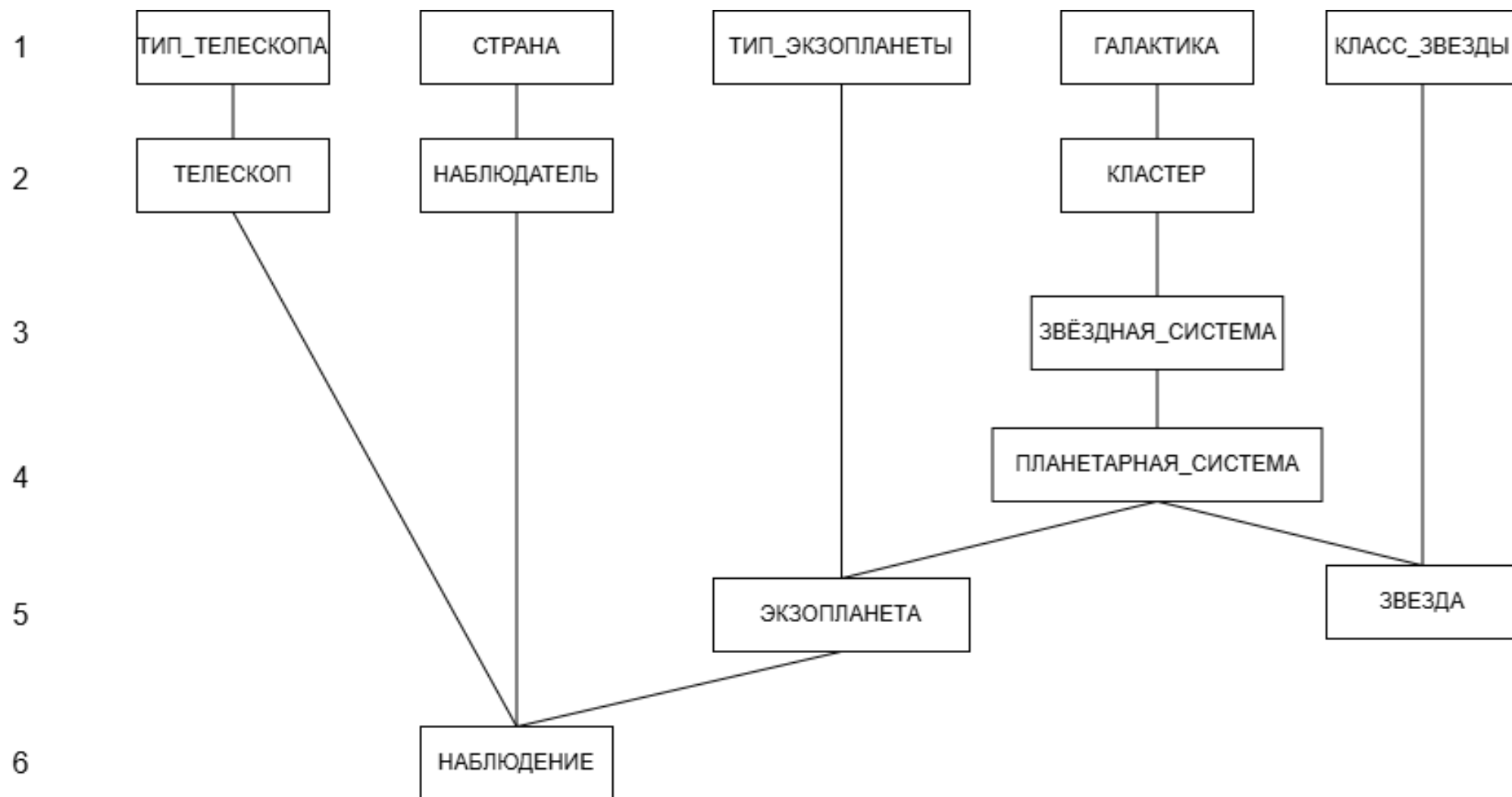


Рис. 5: Диаграмма порядка заполнения таблиц.

3.4. Описание программы генерации данных

Для заполнения базы данных использовался язык программирования Python и библиотека *psycopg2*, предназначенная для подключения к PostgreSQL и выполнения SQL-запросов из Python-программы. Для генерации фиктивных данных применялась стандартная библиотека *random*, а для работы с календарными датами использовались классы *date* и *timedelta* из модуля *datetime*.

Программа выполняет подключение к базе данных, очищает существующие таблицы, генерирует случайное количество записей в диапазоне от 200 000 до 300 000, после чего последовательно заполняет таблицы в порядке, соответствующем иерархии внешних ключей.

```
1 import psycopg2
2 from datetime import date, timedelta
3 import random
4
5 conn = psycopg2.connect(dbname="mydb", user="postgres")
6 cur = conn.cursor()
```

Листинг 1 — Подключение к базе данных

Перед генерацией новых данных выполняется очистка таблиц с помощью оператора *TRUNCATE*. Таблицы перечислены в порядке от наиболее зависимых к родительским, а параметр *CASCADE* позволяет удалить связанные записи, не нарушая ограничения внешних ключей.

```
1 cur.execute("""
2     TRUNCATE TABLE
3         observation,
4         exoplanet,
5         star,
6         telescope,
7         observer,
8         plan_system,
9         stellar_system,
10        cluster,
11        galaxy
12     CASCADE;
13 """)
```

Листинг 2 — Очистка таблиц перед генерацией

Для генерации значений используются вспомогательные функции. Функция *random_date* создаёт случайную дату в заданном диапазоне. Функция *random_decimal* возвращает случайное вещественное число без дополнительного округления в Python. Функция *make_id* формирует строковые идентификаторы объектов по заданному префиксу и номеру.

```
1 def random_date(start_date, end_date):
2     days = (end_date - start_date).days
3     return start_date + timedelta(days=random.randint(0, days))
4
5
6 def random_decimal(min_value, max_value):
7     return random.uniform(min_value, max_value)
8
9
10 def make_id(prefix, number, width=5):
11     return f"{prefix}_{number:0{width}d}"
```

Листинг 3 — Вспомогательные функции генерации

Количество записей в основных таблицах выбирается случайно. При этом программа контролирует итоговый объём данных: сумма записей должна находиться в диапазоне от 200 000 до 300 000. Количество наблюдений оценивается приблизительно как произведение количества экзопланет на среднее число наблюдений для одной экзопланеты.

```
1 TARGET_MIN = 200_000
2 TARGET_MAX = 300_000
3
4 while True:
5     GALAXIES_COUNT = random.randint(30, 70)
6     CLUSTERS_COUNT = random.randint(300, 700)
7     STELLAR_SYSTEMS_COUNT = random.randint(3000, 7000)
8     PLAN_SYSTEMS_COUNT = random.randint(15000, 30000)
9     STARS_COUNT = random.randint(15000, 30000)
10    EXOPLANETS_COUNT = random.randint(30000, 50000)
11    TELESCOPES_COUNT = random.randint(200, 500)
12    OBSERVERS_COUNT = random.randint(7000, 15000)
```

```

13
14     estimated_observations = EXOPLANETS_COUNT * 4
15
16     estimated_total = (
17         GALAXIES_COUNT
18         + CLUSTERS_COUNT
19         + STELLAR_SYSTEMS_COUNT
20         + PLAN_SYSTEMS_COUNT
21         + STARS_COUNT
22         + EXOPLANETS_COUNT
23         + TELESCOPES_COUNT
24         + OBSERVERS_COUNT
25         + estimated_observations
26     )
27
28     if TARGET_MIN <= estimated_total <= TARGET_MAX:
29         break

```

Листинг 4 — Генерация случайного количества записей

После выбора количества записей дополнительно проверяется выполнение ограничений предметной области. В каждом звёздном скоплении должно быть не менее десяти звёзд, не менее одной планетарной системы и не менее одной звёздной системы. Если случайно выбранные значения не удовлетворяют этим условиям, они автоматически увеличиваются до минимально допустимых.

```

1 if STARS_COUNT < CLUSTERS_COUNT * 10:
2     STARS_COUNT = CLUSTERS_COUNT * 10
3
4 if PLAN_SYSTEMS_COUNT < CLUSTERS_COUNT:
5     PLAN_SYSTEMS_COUNT = CLUSTERS_COUNT
6
7 if STELLAR_SYSTEMS_COUNT < CLUSTERS_COUNT:
8     STELLAR_SYSTEMS_COUNT = CLUSTERS_COUNT

```

Листинг 5 — Проверка минимальных требований к иерархии

Заполнение базы данных выполняется в шесть уровней. Сначала задаются справочные значения: типы телескопов, страны, типы экзопланет и

классы звёзд. Так как в данной реализации эти значения представлены перечислениями и списками Python, отдельные INSERT-запросы для них не выполняются. На этом же уровне создаются галактики, так как они являются верхним уровнем космической иерархии и не зависят от других таблиц.

```
1 tele_types = ['space', 'ground']
2
3 exo_types = [
4     'gas_giant',
5     'neptunian',
6     'superearth',
7     'terrestrial'
8 ]
9
10 star_classes = [
11     'O', 'B', 'A', 'F', 'G', 'K', 'M', 'W', 'L', 'D'
12 ]
```

Листинг 6 — Справочные значения

На первом уровне после задания справочных значений заполняется таблица галактик. Для каждой галактики создаётся строковый идентификатор, количество подтверждённых планет, количество планет в зоне обитаемости и расстояние. Количество обитаемых планет генерируется так, чтобы оно не превышало количество подтверждённых планет.

```
1 galaxy_ids = []
2
3 for i in range(1, GALAXIES_COUNT + 1):
4     galaxy_id = make_id("GAL", i, 4)
5
6     conf_plan = random.randint(10, 5000)
7     hab_plan = random.randint(0, conf_plan)
8
9     cur.execute("""
10         INSERT INTO galaxy
11         (galaxy_id, conf_plan, hab_plan, dist)
12         VALUES (%s, %s, %s, %s);
13     """, (
14         galaxy_id,
```

```

15         conf_plan,
16         hab_plan,
17         random_decimal(0, 1000000)
18     ))
19
20     galaxy_ids.append(galaxy_id)

```

Листинг 7 — Вставка галактик

На втором уровне заполняются телескопы, наблюдатели и звёздные скопления. Телескопы и наблюдатели не зависят от космической иерархии, поэтому могут быть созданы до экзопланет и наблюдений. Звёздные скопления создаются после галактик, так как каждое скопление содержит внешний ключ на галактику.

```

1 cur.execute("""
2     INSERT INTO telescope
3     (tele_id, tele_type, aper, com_year, oper, discov_plan)
4     VALUES (%s, %s, %s, %s, %s, %s);
5 """, (
6     tele_id,
7     tele_type,
8     aper,
9     com_year,
10    make_id("OPERATOR", random.randint(1, 9999), 5),
11    0
12 ))

```

Листинг 8 — Вставка телескопа

При создании наблюдателей генерируются идентификатор, дата рождения, страна и начальное количество открытий. Количество открытий изначально равно нулю, а затем пересчитывается после генерации наблюдений.

```

1 cur.execute("""
2     INSERT INTO observer
3     (observer_id, bday, country, discovs)
4     VALUES (%s, %s, %s, %s);
5 """, (
6     observer_id,

```

```

7     random_date(date(1930, 1, 1), date(2005, 12, 31)),
8     random.choice(countries),
9     0
10 ))

```

Листинг 9 — Вставка наблюдателя

При создании звёздных скоплений используется внешний ключ *galaxy_id*. Для этого идентификаторы ранее созданных галактик сохраняются в списке *galaxy_ids*, после чего случайный идентификатор выбирается из этого списка.

```

1 cur.execute("""
2     INSERT INTO cluster
3     (cluster_id, stars_count, conf_plan, hab_plan, dist, galaxy_id)
4     VALUES (%s, %s, %s, %s, %s, %s);
5 """, (
6     cluster_id,
7     random.randint(10, 100000),
8     conf_plan,
9     hab_plan,
10    random_decimal(0, 500000),
11    random.choice(galaxy_ids)
12 ))

```

Листинг 10 — Вставка звёздного скопления

На третьем уровне заполняются звёздные системы. Для выполнения требования о наличии хотя бы одной звёздной системы в каждом скоплении сначала создаётся по одной системе для каждого кластера. Затем остальные звёздные системы распределяются между скоплениями случайным образом. Для дальнейшего использования сохраняются связи между звёздными системами и скоплениями.

```

1 stellar_ids = []
2 stellar_to_cluster = {}
3 cluster_stellar_ids = {cluster_id: [] for cluster_id in cluster_ids}
4
5 stellar_counter = 1

```

```

6
7 for cluster_id in cluster_ids:
8     stellar_id = make_id("STELLAR", stellar_counter, 5)
9     stellar_counter += 1
10
11     cur.execute("""
12         INSERT INTO stellar_system
13         (stellar_id, stars_count, conf_plan, hab_plan, dist,
14          ↪ cluster_id)
15         VALUES (%s, %s, %s, %s, %s, %s);
16     """, (
17         stellar_id,
18         random.randint(1, 8),
19         conf_plan,
20         hab_plan,
21         random_decimal(0, 100000),
22         cluster_id
23     ))
24
25     stellar_ids.append(stellar_id)
26     stellar_to_cluster[stellar_id] = cluster_id
27     cluster_stellar_ids[cluster_id].append(stellar_id)

```

Листинг 11 — Вставка звёздных систем

На четвёртом уровне создаются планетарные системы. Сначала для каждого звёздного скопления создаётся минимум одна планетарная система, связанная с одной из звёздных систем этого скопления. Затем оставшиеся планетарные системы распределяются случайно между уже существующими звёздными системами.

```

1 plan_system_ids = []
2 plan_system_to_cluster = {}
3 cluster_plan_system_ids = {cluster_id: [] for cluster_id in
4 ↪ cluster_ids}
5
6 system_counter = 1
7
8 for cluster_id in cluster_ids:
9     system_id = make_id("SYS", system_counter, 6)
10    system_counter += 1

```

```

10
11     stellar_id = random.choice(cluster_stellar_ids[cluster_id])
12
13     cur.execute("""
14         INSERT INTO plan_system
15         (system_id, conf_plan, hab_plan, distance, stell_sys_id)
16         VALUES (%s, %s, %s, %s, %s);
17     """, (
18         system_id,
19         conf_plan,
20         hab_plan,
21         random_decimal(0, 100000),
22         stellar_id
23     ))
24
25     plan_system_ids.append(system_id)
26     plan_system_to_cluster[system_id] = cluster_id
27     cluster_plan_system_ids[cluster_id].append(system_id)

```

Листинг 12 — Вставка планетарных систем

На пятом уровне заполняются таблицы звёзд и экзопланет. Они находятся на одном уровне иерархии, так как обе таблицы ссылаются на планетарную систему. Для звёзд сначала обеспечивается выполнение требования о наличии минимум десяти звёзд в каждом звёздном скоплении. Для этого звёзды создаются через планетарные системы, принадлежащие соответствующему скоплению.

```

1 for cluster_id in cluster_ids:
2     for _ in range(10):
3         star_id = make_id("STAR", star_counter, 6)
4         star_counter += 1
5
6         system_id = random.choice(cluster_plan_system_ids[cluster_id])
7
8         insert_star(star_id, system_id)
9         star_ids.append(star_id)

```

Листинг 13 — Создание минимум десяти звёзд на скопление

Параметры звезды зависят от её спектрального класса. Для каждого класса задаются собственные диапазоны массы, радиуса и температуры. Это позволяет получать более правдоподобные данные, чем при полностью равномерной генерации всех физических характеристик.

```
1 def generate_star_params():
2     star_class = random.choice(star_classes)
3
4     if star_class == 'O':
5         mass = random_decimal(16, 90)
6         radius = random_decimal(6, 20)
7         temp = random.randint(30000, 50000)
8     elif star_class == 'B':
9         mass = random_decimal(2.1, 16)
10        radius = random_decimal(1.8, 6)
11        temp = random.randint(10000, 30000)
12
13    return star_class, mass, radius, temp
```

Листинг 14 — Генерация параметров звезды

Экзопланеты также генерируются с учётом типа. Для газовых гигантов, нептунowych планет, суперземель и планет земного типа используются разные диапазоны массы и радиуса. Дата последнего наблюдения генерируется так, чтобы она была не раньше даты открытия.

```
1 discov_date = random_date(date(1992, 1, 1), TODAY)
2 last_obs_date = random_date(discov_date, TODAY)
3
4 cur.execute("""
5     INSERT INTO exoplanet
6     (exo_id, mass, radius, orb_period, exo_type, dist,
7     discov_date, last_obs_date, sys_id)
8     VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s);
9 """, (
10     exo_id,
11     mass,
12     radius,
13     random_decimal(0.5, 5000),
14     exo_type,
```

```

15     random_decimal(0, 100000),
16     discov_date,
17     last_obs_date,
18     random.choice(plan_system_ids)
19 ))

```

Листинг 15 — Вставка экзопланеты

На шестом уровне создаются наблюдения. Таблица наблюдений заполняется последней, так как каждая её запись содержит внешние ключи на экзопланету, телескоп и наблюдателя. Для каждой экзопланеты создаётся случайное количество наблюдений от 1 до 7. Чтобы не нарушить ограничение уникальности события наблюдения, комбинация экзопланеты, телескопа, наблюдателя и даты сохраняется в множестве *used_observations*.

```

1 unique_key = (exo_id, tele_id, observer_id, obs_date)
2
3 attempts = 0
4 while unique_key in used_observations and attempts < 20:
5     tele_id = random.choice(tele_ids)
6     observer_id = random.choice(observer_ids)
7     obs_date = random_date(exo_discovery_dates[exo_id], TODAY)
8
9     unique_key = (exo_id, tele_id, observer_id, obs_date)
10    attempts += 1

```

Листинг 16 — Проверка уникальности наблюдения

После создания наблюдений выполняется обновление статистических полей. Для каждого телескопа пересчитывается количество открытых планет, а для каждого наблюдателя — количество открытий. Эти значения накапливаются во время генерации наблюдений и затем записываются в соответствующие таблицы с помощью операторов *UPDATE*.

```

1 for tele_id, count in telescope_discovs.items():
2     cur.execute("""
3         UPDATE telescope
4         SET discov_plan = %s
5         WHERE tele_id = %s;

```

```

6      """ , (
7          count,
8          tele_id
9      ))
10
11 for observer_id, count in observer_discovs.items():
12     cur.execute("""
13         UPDATE observer
14         SET discovs = %s
15         WHERE observer_id = %s;
16     """, (
17         count,
18         observer_id
19     ))

```

Листинг 17 — Обновление статистики телескопов и наблюдателей

Таким образом, порядок заполнения базы данных соответствует следующей иерархии:

1. тип телескопа, страна, тип экзопланеты, класс звезды, галактика;
2. телескоп, наблюдатель, звёздное скопление;
3. звёздная система;
4. планетарная система;
5. звезда, экзопланета;
6. наблюдение.

Такой порядок позволяет сначала сформировать все независимые и родительские сущности, затем создать зависимые космические объекты, а в конце заполнить таблицу наблюдений, которая связывает экзопланеты, телескопы и наблюдателей.

Список литературы

- [1] NASA. Exoplanets - NASA Science [Электронный ресурс] / Chelsea Gohd, Diana Logreira — URL: <https://science.nasa.gov/exoplanets/> (дата обращения: 05.02.2026).
- [2] NASA. What is a Planet? [Электронный ресурс] / Stephen Carney, Diana Logreira — URL: <https://science.nasa.gov/solar-system/planets/what-is-a-planet/> (дата обращения: 05.02.2026).
- [3] ESA. La Silla Observatory [Электронный ресурс] / URL: <https://www.eso.org/public/teles-instr/lasilla/> (дата обращения: 10.02.2026).
- [4] NASA. Exoplanet Catalog [Электронный ресурс] / URL: <https://science.nasa.gov/exoplanets/exoplanet-catalog/> (дата обращения: 10.02.2026).
- [5] <https://www.openexoplanetcatalogue.com/> (дата обращения: 10.02.2026).
- [6] <https://exoplanet.eu/catalog/> (дата обращения: 10.02.2026).
- [7] <https://science.nasa.gov/resource/kepler-186f-the-first-earth-size-planet-in-the-habitable-zone-artists-concept/> (дата обращения: 10.02.2026).
- [8] <https://science.nasa.gov/exoplanets/how-we-find-and-characterize/> (дата обращения: 10.02.2026).
- [9] NASA. In Depth: Exoplanets [Электронный ресурс] / Stephen Carney, Diana Logreira — URL: <https://science.nasa.gov/exoplanets/facts/> (дата обращения: 05.02.2026).
- [10] NASA. What is an Exoplanet? [Электронный ресурс] / Stephen Carney, Diana Logreira — URL: <https://science.nasa.gov/exoplanets/planet-types/> (дата обращения: 05.02.2026).
- [11] <https://science.nasa.gov/mission/webb/>
- [12] NASA. Large Scale Structures - NASA Science [Электронный ресурс] / Jeanette Kazmierczak, Diana Logreira — URL: <https://science.nasa.gov/universe/galaxies/large-scale-structures/> (дата обращения: 07.02.2026).

- [13] Понятие предметной области. Определение сущностей, связей и их свойств. [Электронный ресурс] / URL: <https://studfile.net/preview/9568409/page:26/> (дата обращения: 07.02.2026).
- [14] <https://citizen-science.ru/about/> (дата обращения: 07.02.2026).
- [15] Raghu Ramakrishnan, Johannes Gehrke. Database Management Systems / McGraw-Hill Higher Education — 2 издание — Нью-Йорк, 2002 — 931 с. — 26–27, 136–138 с. — 9780071121132 (дата обращения: 11.02.2026).
- [16] Takahashi, Mana, Azuma, Shoko. The Manga Guide to Databases. / TrendPro CO. LTD — Токио, 2009 — 228 с. — 118 с. — 1593271905 (дата обращения: 08.02.2026).
- [17] NASA. Artist's View of Planetary System in Globular Cluster M4 / URL: <https://science.nasa.gov/asset/hubble/artists-view-of-planetary-system-in-globular-cluster-m4/> (дата обращения: 11.02.2026).
- [18] В.Ю. Кара-Ушанов. Реляционная модель данных / В.Ю. Кара-Ушанов, А.В. Овчинникова — Информационный портал УрФУ — Екатеринбург, 2017 — 6–7 с. (дата обращения: 11.02.2026).
- [19] <https://www.britannica.com/science/stellar-classification>
- [20] <https://science.nasa.gov/universe/exoplanets/discovery-alert-with-six-new-worlds-5500-discovery-milestone-passed/#:~:text=The%20Discovery,-With%20the%20discovery&text=Just%20about%2031%20years%20ago,celebrated%20passing%205%2C000%20exoplanets%20discovered.>
- [21] [https://www.scientificamerican.com/article/how-do-we-name-the-stars/#:~:text=These%20more%20modern%20datasets%20\(and,Betelgeuse.](https://www.scientificamerican.com/article/how-do-we-name-the-stars/#:~:text=These%20more%20modern%20datasets%20(and,Betelgeuse.)

Приложение А. Исходный тест программы генерации базы данных

```
1  -- schema.sql
2
3  -- =====
4  -- ENUM TYPES
5  -- =====
6
7  CREATE TYPE tele_type_enum AS ENUM ('space', 'ground');
8
9  CREATE TYPE exo_type_enum AS ENUM (
10     'gas_giant',
11     'neptunian',
12     'superearth',
13     'terrestrial'
14 );
15
16 CREATE TYPE star_class_enum AS ENUM (
17     'O', 'B', 'A', 'F', 'G', 'K', 'M', 'W', 'L', 'D'
18 );
19
20 CREATE TYPE country_enum AS ENUM (
21     'Afghanistan', 'Albania', 'Algeria', 'Andorra', 'Angola', 'Antigua and
22     ↪ Barbuda',
23     'Argentina', 'Armenia', 'Australia', 'Austria', 'Azerbaijan',
24     'Bahamas', 'Bahrain', 'Bangladesh', 'Barbados', 'Belarus', 'Belgi
25     ↪ ze',
26     'Benin', 'Bhutan', 'Bolivia', 'Bosnia and
27     ↪ Herzegovina', 'Botswana', 'Brazil',
28     'Brunei', 'Bulgaria', 'Burkina Faso', 'Burundi',
29     'Cabo Verde', 'Cambodia', 'Cameroon', 'Canada', 'Central African
30     ↪ Republic',
31     'Chad', 'Chile', 'China', 'Colombia', 'Comoros', 'Congo
32     ↪ (Congo-Brazzaville)',
33     'Costa Rica', 'Croatia', 'Cuba', 'Cyprus', 'Czechia',
34     'Democratic Republic of the Congo', 'Denmark', 'Djibouti', 'Dominica',
35     'Dominican Republic',
36     'Ecuador', 'Egypt', 'El Salvador', 'Equatorial
37     ↪ Guinea', 'Eritrea', 'Estonia',
38     'Eswatini', 'Ethiopia',
```

```

33 'Fiji','Finland','France',
34 'Gabon','Gambia','Georgia','Germany','Ghana','Greece','Grenada',
35 'Guatemala','Guinea','Guinea-Bissau','Guyana',
36 'Haiti','Honduras','Hungary',
37 'Iceland','India','Indonesia','Iran','Iraq','Ireland','Israel','Italy',
   ↪ ' ',
38 'Jamaica','Japan','Jordan',
39 'Kazakhstan','Kenya','Kiribati','Kuwait','Kyrgyzstan',
40 'Laos','Latvia','Lebanon','Lesotho','Liberia','Libya','Liechtenstein',
41 'Lithuania','Luxembourg',
42 'Madagascar','Malawi','Malaysia','Maldives','Mali','Malta',
43 'Marshall Islands','Mauritania','Mauritius','Mexico','Micronesia',
44 'Moldova','Monaco','Mongolia','Montenegro','Morocco','Mozambique',
45 'Myanmar',
46 'Namibia','Nauru','Nepal','Netherlands','New
   ↪ Zealand','Nicaragua','Niger',
47 'Nigeria','North Korea','North Macedonia','Norway',
48 'Oman',
49 'Pakistan','Palau','Panama','Papua New Guinea','Paraguay','Peru',
50 'Philippines','Poland','Portugal',
51 'Qatar',
52 'Romania','Russia','Rwanda',
53 'Saint Kitts and Nevis','Saint Lucia',
54 'Saint Vincent and the Grenadines','Samoa','San Marino',
55 'Sao Tome and Principe','Saudi
   ↪ Arabia','Senegal','Serbia','Seychelles',
56 'Sierra Leone','Singapore','Slovakia','Slovenia','Solomon Islands',
57 'Somalia','South Africa','South Korea','South Sudan','Spain','Sri
   ↪ Lanka',
58 'Sudan','Suriname','Sweden','Switzerland','Syria',
59 'Taiwan','Tajikistan','Tanzania','Thailand','Timor-Leste','Togo','Ton
   ↪ ga',
60 'Trinidad and Tobago','Tunisia','Turkey','Turkmenistan','Tuvalu',
61 'Uganda','Ukraine','United Arab Emirates','United Kingdom',
62 'United States','Uruguay','Uzbekistan',
63 'Vanuatu','Vatican City','Venezuela','Vietnam',
64 'Yemen',
65 'Zambia','Zimbabwe'
66 );
67
68 -- =====
69 -- CORE TABLES

```

```

70 -- =====
71
72 CREATE TABLE galaxy (
73     galaxy_id VARCHAR(30) PRIMARY KEY,
74     conf_plan INT NOT NULL CHECK (conf_plan >= 0),
75     hab_plan  INT NOT NULL CHECK (hab_plan >= 0 AND hab_plan <=
76     ↵ conf_plan),
77     dist      DECIMAL(12, 6) NOT NULL CHECK (dist >= 0)
78 );
79
80 CREATE TABLE cluster (
81     cluster_id  VARCHAR(30) PRIMARY KEY,
82     stars_count INT NOT NULL CHECK (stars_count >= 10),
83     conf_plan   INT NOT NULL CHECK (conf_plan >= 0),
84     hab_plan    INT NOT NULL CHECK (hab_plan >= 0 AND hab_plan <=
85     ↵ conf_plan),
86     dist        DECIMAL(12, 6) NOT NULL CHECK (dist >= 0),
87     galaxy_id   VARCHAR(30) NOT NULL
88     REFERENCES galaxy(galaxy_id) ON DELETE CASCADE
89 );
90
91 CREATE TABLE stellar_system (
92     stellar_id  VARCHAR(30) PRIMARY KEY,
93     stars_count INT NOT NULL CHECK (stars_count >= 1),
94     conf_plan   INT NOT NULL CHECK (conf_plan >= 0),
95     hab_plan    INT NOT NULL CHECK (hab_plan >= 0 AND hab_plan <=
96     ↵ conf_plan),
97     dist        DECIMAL(12, 6) NOT NULL CHECK (dist >= 0),
98     cluster_id  VARCHAR(30) NOT NULL
99     REFERENCES cluster(cluster_id) ON DELETE CASCADE
100 );
101
102 CREATE TABLE plan_system (
103     system_id   VARCHAR(30) PRIMARY KEY,
104     conf_plan   INT NOT NULL CHECK (conf_plan >= 1),
105     hab_plan    INT NOT NULL CHECK (hab_plan >= 0 AND hab_plan <=
106     ↵ conf_plan),
107     distance    DECIMAL(12, 6) NOT NULL CHECK (distance >= 0),
108     stell_sys_id VARCHAR(30) NOT NULL
109     REFERENCES stellar_system(stellar_id) ON DELETE CASCADE
110 );

```

```

108 -- =====
109 -- OBJECTS
110 -- =====
111
112 CREATE TABLE star (
113     star_id VARCHAR(30) PRIMARY KEY,
114     class    star_class_enum NOT NULL,
115     mass     DECIMAL(5, 2) NOT NULL CHECK (mass > 0),
116     radius   DECIMAL(10, 4) NOT NULL CHECK (radius > 0),
117     temp     INT NOT NULL CHECK (temp > 0),
118     dist     DECIMAL(16, 6) NOT NULL CHECK (dist >= 0),
119     sys_id   VARCHAR(30) NOT NULL
120             REFERENCES plan_system(system_id) ON DELETE CASCADE
121 );
122
123 CREATE TABLE exoplanet (
124     exo_id      VARCHAR(30) PRIMARY KEY,
125     mass        DECIMAL(10, 4) NOT NULL CHECK (mass > 0),
126     radius      DECIMAL(10, 4) NOT NULL CHECK (radius > 0),
127     orb_period  DECIMAL(10, 4) NOT NULL CHECK (orb_period > 0),
128     exo_type    exo_type_enum NOT NULL,
129     dist        DECIMAL(12, 6) NOT NULL CHECK (dist >= 0),
130     discov_date DATE NOT NULL CHECK (
131         discov_date BETWEEN DATE '1992-01-01' AND CURRENT_DATE
132     ),
133     last_obs_date DATE NOT NULL CHECK (
134         last_obs_date BETWEEN discov_date AND CURRENT_DATE
135     ),
136     sys_id      VARCHAR(30) NOT NULL
137             REFERENCES plan_system(system_id) ON DELETE CASCADE
138 );
139
140 CREATE TABLE telescope (
141     tele_id VARCHAR(47) PRIMARY KEY,
142     tele_type tele_type_enum NOT NULL,
143     aper DECIMAL(6,2) NOT NULL CHECK (aper > 0),
144     com_year INT NOT NULL CHECK (
145         com_year BETWEEN 1609 AND EXTRACT(YEAR FROM CURRENT_DATE)::INT
146     ),
147     oper VARCHAR(81) NOT NULL,
148     discov_plan INT NOT NULL CHECK (discov_plan >= 0)
149 );

```

```

150
151 CREATE TABLE observer (
152     observer_id VARCHAR(100) PRIMARY KEY,
153     bday         DATE NOT NULL CHECK (
154         bday BETWEEN DATE '1609-01-01' AND CURRENT_DATE
155     ),
156     country      country_enum NOT NULL,
157     discovs      INT NOT NULL CHECK (discovs >= 0)
158 );
159
160 CREATE TABLE observation (
161     observation_id VARCHAR(30) PRIMARY KEY,
162     exo_id          VARCHAR(30) NOT NULL
163         REFERENCES exoplanet(exo_id) ON DELETE CASCADE,
164     tele_id         VARCHAR(47) NOT NULL
165         REFERENCES telescope(tele_id) ON DELETE CASCADE,
166     observer_id     VARCHAR(100) NOT NULL
167         REFERENCES observer(observer_id) ON DELETE CASCADE,
168     obs_date        DATE NOT NULL CHECK (
169         obs_date BETWEEN DATE '1992-01-01' AND CURRENT_DATE
170     ),
171     CONSTRAINT observation_unique_event
172         UNIQUE (exo_id, tele_id, observer_id, obs_date)
173 );

```

Приложение Б. Исходный текст программы генерации тестовых данных

```
1 import psycopg2
2 from datetime import date, timedelta
3 import random
4
5 conn = psycopg2.connect(dbname="mydb", user="postgres")
6 cur = conn.cursor()
7
8 print("Очистка таблиц...")
9
10 cur.execute("""
11     TRUNCATE TABLE
12         observation,
13         exoplanet,
14         star,
15         telescope,
16         observer,
17         plan_system,
18         stellar_system,
19         cluster,
20         galaxy
21     CASCADE;
22 """)
23
24 TODAY = date.today()
25
26
27 def random_date(start_date, end_date):
28     days = (end_date - start_date).days
29     return start_date + timedelta(days=random.randint(0, days))
30
31
32 def random_decimal(min_value, max_value):
33     return random.uniform(min_value, max_value)
34
35
36 def make_id(prefix, number, width=5):
37     return f"{prefix}_{number:0{width}d}"
38
```

```

39
40 \# Случайное количество записей
41 \# с контролем общей суммы 200k-300k
42 \# =====
43 TARGET_MIN = 200_000
44 TARGET_MAX = 300_000
45
46 while True:
47     GALAXIES_COUNT = random.randint(30, 70)
48     CLUSTERS_COUNT = random.randint(300, 700)
49     STELLAR_SYSTEMS_COUNT = random.randint(3000, 7000)
50     PLAN_SYSTEMS_COUNT = random.randint(15000, 30000)
51     STARS_COUNT = random.randint(25000, 50000)
52     EXOPLANETS_COUNT = random.randint(30000, 50000)
53     TELESCOPES_COUNT = random.randint(200, 500)
54     OBSERVERS_COUNT = random.randint(7000, 15000)
55
56     estimated_observations = EXOPLANETS_COUNT * 4
57
58     estimated_total = (
59         GALAXIES_COUNT
60         + CLUSTERS_COUNT
61         + STELLAR_SYSTEMS_COUNT
62         + PLAN_SYSTEMS_COUNT
63         + STARS_COUNT
64         + EXOPLANETS_COUNT
65         + TELESCOPES_COUNT
66         + OBSERVERS_COUNT
67         + estimated_observations
68     )
69
70     if TARGET_MIN <= estimated_total <= TARGET_MAX:
71         break
72
73 \# Гарантия, что хватит звёзд: минимум 10 звёзд на каждое скопление
74 if STARS_COUNT < CLUSTERS_COUNT * 10:
75     STARS_COUNT = CLUSTERS_COUNT * 10
76
77 \# Гарантия, что хватит планетарных систем: минимум 1 на каждое
78 ↪ скопление
79 if PLAN_SYSTEMS_COUNT < CLUSTERS_COUNT:
80     PLAN_SYSTEMS_COUNT = CLUSTERS_COUNT

```

```

80
81 \# Гарантия, что хватит звёздных систем: минимум 1 на каждое скопление
82 if STELLAR_SYSTEMS_COUNT < CLUSTERS_COUNT:
83     STELLAR_SYSTEMS_COUNT = CLUSTERS_COUNT
84
85 print("Случайно выбранное количество записей:")
86 print(f"Галактик: {GALAXIES_COUNT}")
87 print(f"Скоплений: {CLUSTERS_COUNT}")
88 print(f"Звёздных систем: {STELLAR_SYSTEMS_COUNT}")
89 print(f"Планетных систем: {PLAN_SYSTEMS_COUNT}")
90 print(f"Звёзд: {STARS_COUNT}")
91 print(f"Экзопланет: {EXOPLANETS_COUNT}")
92 print(f"Телескопов: {TELESCOPES_COUNT}")
93 print(f"Наблюдателей: {OBSERVERS_COUNT}")
94 print(f"Примерная сумма: {estimated_total}")
95
96 \# =====
97 \# ENUM значения
98 \# =====
99 exo_types = [
100     'gas_giant',
101     'neptunian',
102     'superearth',
103     'terrestrial'
104 ]
105
106 star_classes = [
107     'O', 'B', 'A', 'F', 'G', 'K', 'M', 'W', 'L', 'D'
108 ]
109
110 tele_types = [
111     'space',
112     'ground'
113 ]
114
115 countries = [
116     'Afghanistan', 'Albania', 'Algeria', 'Andorra', 'Angola', 'Antigua and
↵ Barbuda',
117     'Argentina', 'Armenia', 'Australia', 'Austria', 'Azerbaijan',
118     'Bahamas', 'Bahrain', 'Bangladesh', 'Barbados', 'Belarus', 'Belgi
↵ ze',

```

119 'Benin', 'Bhutan', 'Bolivia', 'Bosnia and
 ↳ Herzegovina', 'Botswana', 'Brazil',
 120 'Brunei', 'Bulgaria', 'Burkina Faso', 'Burundi',
 121 'Cabo Verde', 'Cambodia', 'Cameroon', 'Canada', 'Central African
 ↳ Republic',
 122 'Chad', 'Chile', 'China', 'Colombia', 'Comoros', 'Congo
 ↳ (Congo-Brazzaville)',
 123 'Costa Rica', 'Croatia', 'Cuba', 'Cyprus', 'Czechia',
 124 'Democratic Republic of the Congo', 'Denmark', 'Djibouti', 'Dominica',
 125 'Dominican Republic',
 126 'Ecuador', 'Egypt', 'El Salvador', 'Equatorial
 ↳ Guinea', 'Eritrea', 'Estonia',
 127 'Eswatini', 'Ethiopia',
 128 'Fiji', 'Finland', 'France',
 129 'Gabon', 'Gambia', 'Georgia', 'Germany', 'Ghana', 'Greece', 'Grenada',
 130 'Guatemala', 'Guinea', 'Guinea-Bissau', 'Guyana',
 131 'Haiti', 'Honduras', 'Hungary',
 132 'Iceland', 'India', 'Indonesia', 'Iran', 'Iraq', 'Ireland', 'Israel', 'Italy',
 ↳ '
 133 'Jamaica', 'Japan', 'Jordan',
 134 'Kazakhstan', 'Kenya', 'Kiribati', 'Kuwait', 'Kyrgyzstan',
 135 'Laos', 'Latvia', 'Lebanon', 'Lesotho', 'Liberia', 'Libya', 'Liechtenstein',
 136 'Lithuania', 'Luxembourg',
 137 'Madagascar', 'Malawi', 'Malaysia', 'Maldives', 'Mali', 'Malta',
 138 'Marshall Islands', 'Mauritania', 'Mauritius', 'Mexico', 'Micronesia',
 139 'Moldova', 'Monaco', 'Mongolia', 'Montenegro', 'Morocco', 'Mozambique',
 140 'Myanmar',
 141 'Namibia', 'Nauru', 'Nepal', 'Netherlands', 'New
 ↳ Zealand', 'Nicaragua', 'Niger',
 142 'Nigeria', 'North Korea', 'North Macedonia', 'Norway',
 143 'Oman',
 144 'Pakistan', 'Palau', 'Panama', 'Papua New Guinea', 'Paraguay', 'Peru',
 145 'Philippines', 'Poland', 'Portugal',
 146 'Qatar',
 147 'Romania', 'Russia', 'Rwanda',
 148 'Saint Kitts and Nevis', 'Saint Lucia',
 149 'Saint Vincent and the Grenadines', 'Samoa', 'San Marino',
 150 'Sao Tome and Principe', 'Saudi
 ↳ Arabia', 'Senegal', 'Serbia', 'Seychelles',
 151 'Sierra Leone', 'Singapore', 'Slovakia', 'Slovenia', 'Solomon Islands',
 152 'Somalia', 'South Africa', 'South Korea', 'South Sudan', 'Spain', 'Sri
 ↳ Lanka',

```

153 'Sudan', 'Suriname', 'Sweden', 'Switzerland', 'Syria',
154 'Taiwan', 'Tajikistan', 'Tanzania', 'Thailand', 'Timor-Leste', 'Togo', 'Ton
    ↪ ga',
155 'Trinidad and Tobago', 'Tunisia', 'Turkey', 'Turkmenistan', 'Tuvalu',
156 'Uganda', 'Ukraine', 'United Arab Emirates', 'United Kingdom',
157 'United States', 'Uruguay', 'Uzbekistan',
158 'Vanuatu', 'Vatican City', 'Venezuela', 'Vietnam',
159 'Yemen',
160 'Zambia', 'Zimbabwe'
161 ]
162
163 operators = [
164     'NASA',
165     'ESA',
166     'Roscosmos',
167     'JAXA',
168     'CNSA',
169     'ESO',
170     'ISRO',
171     'DLR',
172     'CNES',
173     'Private Observatory Network',
174     'International Space Research Group',
175     'Global Telescope Association'
176 ]
177
178 \# =====
179 \# 1. Галактики
180 \# =====
181 print("Галактики...")
182
183 galaxy_ids = []
184
185 for i in range(1, GALAXIES_COUNT + 1):
186     galaxy_id = make_id("GAL", i, 4)
187
188     conf_plan = random.randint(10, 5000)
189     hab_plan = random.randint(0, conf_plan)
190
191     cur.execute("""
192         INSERT INTO galaxy
193         (galaxy_id, conf_plan, hab_plan, dist)

```

```

194         VALUES (%s, %s, %s, %s);
195     """ , (
196         galaxy_id,
197         conf_plan,
198         hab_plan,
199         random_decimal(0, 1000000)
200     ))
201
202     galaxy_ids.append(galaxy_id)
203
204     \# =====
205     \# 2. Скопления
206     \# =====
207     print("Скопления...")
208
209     cluster_ids = []
210
211     for i in range(1, CLUSTERS_COUNT + 1):
212         cluster_id = make_id("CLS", i, 5)
213
214         conf_plan = random.randint(5, 2000)
215         hab_plan = random.randint(0, conf_plan)
216
217         cur.execute("""
218             INSERT INTO cluster
219             (cluster_id, stars_count, conf_plan, hab_plan, dist, galaxy_id)
220             VALUES (%s, %s, %s, %s, %s, %s);
221         """ , (
222             cluster_id,
223             random.randint(10, 100000),
224             conf_plan,
225             hab_plan,
226             random_decimal(0, 500000),
227             random.choice(galaxy_ids)
228         ))
229
230         cluster_ids.append(cluster_id)
231
232     \# =====
233     \# 3. Звёздные системы
234     \# =====
235     print("Звёздные системы...")

```

```

236
237 stellar_ids = []
238 stellar_to_cluster = {}
239 cluster_stellar_ids = {cluster_id: [] for cluster_id in cluster_ids}
240
241 stellar_counter = 1
242
243 \# Минимум 1 звёздная система на каждое скопление
244 for cluster_id in cluster_ids:
245     stellar_id = make_id("STELLAR", stellar_counter, 5)
246     stellar_counter += 1
247
248     conf_plan = random.randint(1, 300)
249     hab_plan = random.randint(0, conf_plan)
250
251     cur.execute("""
252         INSERT INTO stellar_system
253         (stellar_id, stars_count, conf_plan, hab_plan, dist,
254           ↪ cluster_id)
255         VALUES (%s, %s, %s, %s, %s, %s);
256     """, (
257         stellar_id,
258         random.randint(1, 8),
259         conf_plan,
260         hab_plan,
261         random_decimal(0, 100000),
262         cluster_id
263     ))
264
265     stellar_ids.append(stellar_id)
266     stellar_to_cluster[stellar_id] = cluster_id
267     cluster_stellar_ids[cluster_id].append(stellar_id)
268
269 \# Остальные звёздные системы случайно
270 while stellar_counter <= STELLAR_SYSTEMS_COUNT:
271     stellar_id = make_id("STELLAR", stellar_counter, 5)
272     stellar_counter += 1
273
274     cluster_id = random.choice(cluster_ids)
275
276     conf_plan = random.randint(1, 300)
277     hab_plan = random.randint(0, conf_plan)

```

```

277
278 cur.execute("""
279     INSERT INTO stellar_system
280     (stellar_id, stars_count, conf_plan, hab_plan, dist,
281      ↪ cluster_id)
282     VALUES (%s, %s, %s, %s, %s, %s);
283 """, (
284     stellar_id,
285     random.randint(1, 8),
286     conf_plan,
287     hab_plan,
288     random_decimal(0, 100000),
289     cluster_id
290 ))
291
292 stellar_ids.append(stellar_id)
293 stellar_to_cluster[stellar_id] = cluster_id
294 cluster_stellar_ids[cluster_id].append(stellar_id)
295
296 \# =====
297 \# 4. Планетные системы
298 \# =====
299 print("Планетные системы...")
300
301 plan_system_ids = []
302 plan_system_to_cluster = {}
303 cluster_plan_system_ids = {cluster_id: [] for cluster_id in
304 ↪ cluster_ids}
305
306 system_counter = 1
307
308 \# Минимум 1 планетарная система на каждое звёздное скопление
309 for cluster_id in cluster_ids:
310     system_id = make_id("SYS", system_counter, 6)
311     system_counter += 1
312
313     stellar_id = random.choice(cluster_stellar_ids[cluster_id])
314
315     conf_plan = random.randint(1, 20)
316     hab_plan = random.randint(0, conf_plan)
317
318 cur.execute("""

```

```

317         INSERT INTO plan_system
318         (system_id, conf_plan, hab_plan, distance, stell_sys_id)
319         VALUES (%s, %s, %s, %s, %s);
320     """ , (
321         system_id,
322         conf_plan,
323         hab_plan,
324         random_decimal(0, 100000),
325         stellar_id
326     ))
327
328     plan_system_ids.append(system_id)
329     plan_system_to_cluster[system_id] = cluster_id
330     cluster_plan_system_ids[cluster_id].append(system_id)
331
332     \# Остальные планетарные системы случайно
333     while system_counter <= PLAN_SYSTEMS_COUNT:
334         system_id = make_id("SYS", system_counter, 6)
335         system_counter += 1
336
337         stellar_id = random.choice(stellar_ids)
338         cluster_id = stellar_to_cluster[stellar_id]
339
340         conf_plan = random.randint(1, 20)
341         hab_plan = random.randint(0, conf_plan)
342
343         cur.execute("""
344             INSERT INTO plan_system
345             (system_id, conf_plan, hab_plan, distance, stell_sys_id)
346             VALUES (%s, %s, %s, %s, %s);
347         """ , (
348             system_id,
349             conf_plan,
350             hab_plan,
351             random_decimal(0, 100000),
352             stellar_id
353         ))
354
355         plan_system_ids.append(system_id)
356         plan_system_to_cluster[system_id] = cluster_id
357         cluster_plan_system_ids[cluster_id].append(system_id)
358

```

```

359 \# =====
360 \# 5. Звёзды
361 \# =====
362 print("Звёзды...")
363
364
365 def generate_star_params():
366     star_class = random.choice(star_classes)
367
368     if star_class == 'O':
369         mass = random_decimal(16, 90)
370         radius = random_decimal(6, 20)
371         temp = random.randint(30000, 50000)
372
373     elif star_class == 'B':
374         mass = random_decimal(2.1, 16)
375         radius = random_decimal(1.8, 6)
376         temp = random.randint(10000, 30000)
377
378     elif star_class == 'A':
379         mass = random_decimal(1.4, 2.1)
380         radius = random_decimal(1.4, 2.5)
381         temp = random.randint(7500, 10000)
382
383     elif star_class == 'F':
384         mass = random_decimal(1.04, 1.4)
385         radius = random_decimal(1.15, 1.4)
386         temp = random.randint(6000, 7500)
387
388     elif star_class == 'G':
389         mass = random_decimal(0.8, 1.04)
390         radius = random_decimal(0.9, 1.15)
391         temp = random.randint(5200, 6000)
392
393     elif star_class == 'K':
394         mass = random_decimal(0.45, 0.8)
395         radius = random_decimal(0.7, 0.9)
396         temp = random.randint(3700, 5200)
397
398     elif star_class == 'M':
399         mass = random_decimal(0.08, 0.45)
400         radius = random_decimal(0.1, 0.7)

```

```

401         temp = random.randint(2400, 3700)
402
403     else:
404         mass = random_decimal(0.1, 10)
405         radius = random_decimal(0.01, 5)
406         temp = random.randint(1000, 100000)
407
408     return star_class, mass, radius, temp
409
410
411 def insert_star(star_id, system_id):
412     star_class, mass, radius, temp = generate_star_params()
413
414     cur.execute("""
415         INSERT INTO star
416         (star_id, class, mass, radius, temp, dist, sys_id)
417         VALUES (%s, %s, %s, %s, %s, %s, %s);
418     """, (
419         star_id,
420         star_class,
421         mass,
422         radius,
423         temp,
424         random_decimal(0, 100000),
425         system_id
426     ))
427
428
429 star_ids = []
430 star_counter = 1
431
432 \# Минимум 10 звёзд в каждом звёздном скоплении
433 for cluster_id in cluster_ids:
434     for _ in range(10):
435         star_id = make_id("STAR", star_counter, 6)
436         star_counter += 1
437
438         system_id = random.choice(cluster_plan_system_ids[cluster_id])
439
440         insert_star(star_id, system_id)
441         star_ids.append(star_id)
442

```

```

443 \# Остальные звёзды случайно
444 while star_counter <= STARS_COUNT:
445     star_id = make_id("STAR", star_counter, 6)
446     star_counter += 1
447
448     system_id = random.choice(plan_system_ids)
449
450     insert_star(star_id, system_id)
451     star_ids.append(star_id)
452
453 \# =====
454 \# 6. Экзопланеты
455 \# =====
456 print("Экзопланеты...")
457
458 exo_ids = []
459 exo_discovery_dates = {}
460
461 for i in range(1, EXOPLANETS_COUNT + 1):
462     exo_id = make_id("EXO", i, 6)
463     exo_type = random.choice(exo_types)
464
465     if exo_type == 'gas_giant':
466         mass = random_decimal(50, 5000)
467         radius = random_decimal(5, 20)
468
469     elif exo_type == 'neptunian':
470         mass = random_decimal(10, 50)
471         radius = random_decimal(2, 6)
472
473     elif exo_type == 'superearth':
474         mass = random_decimal(2, 10)
475         radius = random_decimal(1.2, 2.5)
476
477     else:
478         mass = random_decimal(0.1, 2)
479         radius = random_decimal(0.3, 1.5)
480
481     discov_date = random_date(date(1992, 1, 1), TODAY)
482     last_obs_date = random_date(discov_date, TODAY)
483
484     cur.execute("""

```

```

485         INSERT INTO exoplanet
486         (exo_id, mass, radius, orb_period, exo_type, dist,
487          discov_date, last_obs_date, sys_id)
488         VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s);
489     """ , (
490         exo_id,
491         mass,
492         radius,
493         random_decimal(0.5, 5000),
494         exo_type,
495         random_decimal(0, 100000),
496         discov_date,
497         last_obs_date,
498         random.choice(plan_system_ids)
499     ))
500
501     exo_ids.append(exo_id)
502     exo_discovery_dates[exo_id] = discov_date
503
504     \# =====
505     \# 7. Телескопы
506     \# =====
507     print("Телескопы...")
508
509     tele_ids = []
510
511     for i in range(1, TELESCOPES_COUNT + 1):
512         tele_id = make_id("TELE", i, 5)
513
514         tele_type = random.choice(tele_types)
515
516         if tele_type == 'space':
517             aper = random_decimal(0.2, 10)
518             com_year = random.randint(1960, TODAY.year)
519         else:
520             aper = random_decimal(0.5, 40)
521             com_year = random.randint(1609, TODAY.year)
522
523         cur.execute("""
524             INSERT INTO telescope
525             (tele_id, tele_type, aper, com_year, oper, discov_plan)
526             VALUES (%s, %s, %s, %s, %s, %s);

```

```

527         """", (
528             tele_id,
529             tele_type,
530             aper,
531             com_year,
532             random.choice(operators),
533             0
534         ))
535
536     tele_ids.append(tele_id)
537
538     \# =====
539     \# 8. Наблюдатели
540     \# =====
541     print("Наблюдатели...")
542
543     observer_ids = []
544
545     for i in range(1, OBSERVERS_COUNT + 1):
546         observer_id = f"Observer_{i:06d}"
547
548         cur.execute("""
549             INSERT INTO observer
550             (observer_id, bday, country, discovs)
551             VALUES (%s, %s, %s, %s);
552         """, (
553             observer_id,
554             random_date(date(1930, 1, 1), date(2005, 12, 31)),
555             random.choice(countries),
556             0
557         ))
558
559         observer_ids.append(observer_id)
560
561     \# =====
562     \# 9. Наблюдения
563     \# =====
564     print("Наблюдения...")
565
566     observation_ids = []
567     used_observations = set()
568

```

```

569 telescope_discovs = {tele_id: 0 for tele_id in tele_ids}
570 observer_discovs = {observer_id: 0 for observer_id in observer_ids}
571
572 obs_counter = 1
573
574 for exo_id in exo_ids:
575     observations_for_planet = random.randint(1, 7)
576
577     for _ in range(observations_for_planet):
578         tele_id = random.choice(tele_ids)
579         observer_id = random.choice(observer_ids)
580
581         obs_date = random_date(exo_discovery_dates[exo_id], TODAY)
582
583         unique_key = (exo_id, tele_id, observer_id, obs_date)
584
585         attempts = 0
586         while unique_key in used_observations and attempts < 20:
587             tele_id = random.choice(tele_ids)
588             observer_id = random.choice(observer_ids)
589             obs_date = random_date(exo_discovery_dates[exo_id], TODAY)
590
591             unique_key = (exo_id, tele_id, observer_id, obs_date)
592             attempts += 1
593
594         if unique_key in used_observations:
595             continue
596
597         used_observations.add(unique_key)
598
599         observation_id = make_id("OBS", obs_counter, 7)
600         obs_counter += 1
601
602         cur.execute("""
603             INSERT INTO observation
604             (observation_id, exo_id, tele_id, observer_id, obs_date)
605             VALUES (%s, %s, %s, %s, %s);
606         """, (
607             observation_id,
608             exo_id,
609             tele_id,
610             observer_id,

```

```

611         obs_date
612     ))
613
614     observation_ids.append(observation_id)
615
616     telescope_discovs[tele_id] += 1
617     observer_discovs[observer_id] += 1
618
619     \# =====
620     \# 10. Обновление статистики
621     \# =====
622     print("Обновление статистики...")
623
624     for tele_id, count in telescope_discovs.items():
625         cur.execute("""
626             UPDATE telescope
627             SET discov_plan = %s
628             WHERE tele_id = %s;
629         """, (
630             count,
631             tele_id
632         ))
633
634     for observer_id, count in observer_discovs.items():
635         cur.execute("""
636             UPDATE observer
637             SET discovs = %s
638             WHERE observer_id = %s;
639         """, (
640             count,
641             observer_id
642         ))
643
644     \# =====
645     \# Завершение
646     \# =====
647     conn.commit()
648     cur.close()
649     conn.close()
650
651     total_records = (
652         len(galaxy_ids)

```

```

653     + len(cluster_ids)
654     + len(stellar_ids)
655     + len(plan_system_ids)
656     + len(star_ids)
657     + len(exo_ids)
658     + len(tele_ids)
659     + len(observer_ids)
660     + len(observation_ids)
661 )
662
663 print("Готово!")
664 print(f"Галактик: {len(galaxy_ids)}")
665 print(f"Скоплений: {len(cluster_ids)}")
666 print(f"Звёздных систем: {len(stellar_ids)}")
667 print(f"Планетных систем: {len(plan_system_ids)}")
668 print(f"Звёзд: {len(star_ids)}")
669 print(f"Экзопланет: {len(exo_ids)}")
670 print(f"Телескопов: {len(tele_ids)}")
671 print(f"Наблюдателей: {len(observer_ids)}")
672 print(f"Наблюдений: {len(observation_ids)}")
673 print(f"Всего записей: {total_records}")
674
675 \# print()
676 \# print("Проверочные SQL-запросы:")
677 \# print("")
678 \# 1. Проверка, что в каждом скоплении есть минимум 1 планетарная
    ↪ система:
679
680 \# SELECT
681 \#     c.cluster_id,
682 \#     COUNT(DISTINCT ps.system_id) AS planet_systems_count
683 \# FROM cluster c
684 \# LEFT JOIN stellar_system ss
685 \#     ON ss.cluster_id = c.cluster_id
686 \# LEFT JOIN plan_system ps
687 \#     ON ps.stell_sys_id = ss.stellar_id
688 \# GROUP BY c.cluster_id
689 \# HAVING COUNT(DISTINCT ps.system_id) < 1;
690
691 \# 2. Проверка, что в каждом скоплении есть минимум 10 звёзд:
692
693 \# SELECT

```

```
694 \#      c.cluster_id,  
695 \#      COUNT(s.star_id) AS stars_count  
696 \# FROM cluster c  
697 \# LEFT JOIN stellar_system ss  
698 \#      ON ss.cluster_id = c.cluster_id  
699 \# LEFT JOIN plan_system ps  
700 \#      ON ps.stell_sys_id = ss.stellar_id  
701 \# LEFT JOIN star s  
702 \#      ON s.sys_id = ps.system_id  
703 \# GROUP BY c.cluster_id  
704 \# HAVING COUNT(s.star_id) < 10;  
705 \# """)
```